Failure Attribution in LLM Agent Systems

A working definition and four research proposals

Companion to the dataset survey *Datasets for Failure Attribution in LLM Agent Systems*, 30 October 2026

Scope. This document fixes a working definition of failure attribution as an interventional quantity and develops four research proposals that follow from it. It is the companion of the survey `main.pdf`, which describes the 21 datasets the proposals draw their evidence from. References of the form §3.2, Fig. 4, Section 7 or Table 1 that do not resolve inside this document point into the survey; in the PDF they link to `main.pdf`. All data, notes and scripts are in `/dataDisk/home/thanhdo/failure-attribution-datasets/`.

Contents

1 Overview 1

2 Preliminaries: a working definition of failure attribution 1

3 Proposal 1: Counterfactual perturbation screens on agent trajectories 3

4 Proposal 2: Label-free attribution from the flow of trajectories 5

5 Proposal 3: Attribution-guided policy improvement 8

6 Proposal 4: Error propagation and absorption—the lifecycle of an error 11

1 Overview

What this document contains. The survey leaves one definitional gap and four research questions behind it. The gap is treated first; the four proposals follow and build on each other.

- **Preliminaries** (Section 2) fix a working definition of failure attribution as an *interventional* quantity, together with the two assumptions it needs and the data engine that every proposal uses.
- **Proposal 1** (Section 3) *measures* that quantity with counterfactual rollouts and learns to predict it without executing.
- **Proposal 2** (Section 4) *estimates* it from logs and outcomes alone, with no step label, through a hazard model.
- **Proposal 3** (Section 5) *uses* it as a training signal for the agent that produced the failure and asks how much attribution accuracy is worth.
- **Proposal 4** (Section 6) looks *between* the error and the outcome: how an error propagates, is absorbed or locks in, which is the regime the first three treat as a point event.

How the proposals are written. Every proposal has the same nine parts, in the same order, so that they can be read side by side:

- *Motivation*: the evidence from the surveyed data that opens the question;
- *Closest prior work*: what already exists and what it lacks;
- *Setting*: the objects the proposal needs, introduced where they are first used as boxed **definitions** and **assumptions**;
- *Research questions*: stated as full sentences in a **purple box** and numbered **RQ** $p.i$ for proposal p;
- *Experiments*: numbered **E** $p.j$, each naming the research questions it answers, fully or in part;
- *Evaluation*: one table per proposal giving, for each experiment, the data (collected or released), the metrics and the baselines or reference values;
- *Feasibility with the surveyed data*, *Pitfalls to design for* and *What would count as a result*.

Table 1 places the four proposals on one line each.

Proposal	Question in one sentence	Supervision it needs	What it produces
1 Perturbation screens	How much does the outcome change when one step is repaired or corrupted, and can that change be predicted without running the system?	replayable environment, no human label	effect profiles e_τ, fragility curves, a surrogate f
2 Label-free hazard model	Where in a failed run is the fatal step, when only outcomes—or nothing—are known?	outcome labels or none	a step-hazard model h_θ, an online monitor
3 Attribution as training signal	Does a better attributor make a better agent, and at what accuracy does it start to help?	a trainable policy and a replayable environment	the accuracy-to-value curve, culprit-only updates
4 Error lifecycle	When an error enters a run, is it detected, corrected, tolerated or fatal, and how fast?	injections with absorbed branches kept	fate distributions, lifetime curves, per-agent profiles

Table 1: The four proposals. All of them measure or estimate the same object—the success probability of a prefix and how an intervention moves it—and all run on the data engine of Table 3.

2 Preliminaries: a working definition of failure attribution

Why a definition is needed. The four proposals need a target that stays fixed from one dataset to the next. The survey shows that the current targets do not:

- Every dataset defines its labelled step *counterfactually*—"the earliest step such that, had it been correct, the run would have succeeded"—but only TraceElephant ever tests the counterfactual by re-execution (§3.2).
- The summary that is labelled differs: the earliest decisive step (Who&When, TrajErrBench, LongRCA), the first *unrecovered* failure (AgentRx), a set of steps with per-annotator votes (MP-Bench), every

error span (TRAIL), three timestamps $t_{\text{err}}/t_{\text{lock}}/t_{\text{obs}}$ (Failure-as-a-Process), a step with a tolerance span (SearchAuditBench) or a set of (edge, fault side, mode) triples (MegaRCA-Mix).

- Humans do not agree on the proxy: exact-step agreement is 40.3% in LongRCA, 77.9% in AgentHallu and 91.2% / 66.7% on the two TrajErrBench domains; TraceElephant's round-1 Krippendorff's α for the step is 0.64; the blamed agent agrees between Who&When and TraceElephant on only 18 of 86 paired tasks.

We therefore take the counterfactual itself as the definition and treat every human label as a noisy estimate of it.

Setting and notation. Four objects are used throughout the document.

- An agent system π (single- or multi-agent, stochastic) interacts with an environment $\mathcal{E}$ on a task x with an automatic success predicate $S \in \{0, 1\}$ (ground-truth answer, unit tests, verifier).
- A trajectory is $\tau = (s_1, \ldots, s_T)$; step $s_t = (a_t, u_t, o_t)$ records the acting agent $a_t \in \mathcal{A}$, its output u_t (message, tool call or code) and the resulting observation o_t (tool result, page, test log). $\tau_{<k}$ is the prefix $(s_1, \ldots, s_{k-1})$.
- *One step is one policy decision*: an agent output together with the observation it produced. Orchestrator bookkeeping, user turns and bare tool returns are context, not candidates; this removes the indexing differences catalogued in Section 7.
- $\text{Roll}(\tau_{<k} \oplus s'_k)$ is the random continuation obtained by restoring the state reached after $\tau_{<k}$, executing the (possibly modified) step s'_k and letting π and $\mathcal{E}$ run to termination; randomness comes from sampling in π and from $\mathcal{E}$.

Assumption 1 (Replayability)

The environment state after $\tau_{<k}$ can be restored exactly (container snapshot, sandbox seed, simulated user), so that $\text{Roll}(\cdot)$ is a well-defined distribution over continuations.

- Live-web environments violate this. SymTrace's selective replay [39] serves the prefix from recorded tool returns, which restores the prefix exactly but leaves the continuation unreproducible; the authors note that temperature-0 calls to a hosted model do not guarantee determinism either.
- Live-web cases are therefore excluded from the measured part of every proposal and used for transfer tests only.

Assumption 2 (Fixed intervention vocabulary)

Interventions come from a finite, declared set $\Delta = \Delta_{\text{fix}} \cup \Delta_{\text{inj}}$; each $\delta \in \Delta$ is a deterministic map $s_k \mapsto \delta(s_k)$ that changes exactly one step. Table 2 lists a starting vocabulary. "A change to a step" without such a vocabulary is not a learnable object.

Definition 1 (Intervention effect)

For a trajectory τ, a step index k and an intervention $\delta \in \Delta$, the *baseline* and *intervened success rates* are

$$p_0(\tau, k) = \Pr\left[S(\text{Roll}(\tau_{<k} \oplus s_k)) = 1\right],$$
$$p_\delta(\tau, k) = \Pr\left[S(\text{Roll}(\tau_{<k} \oplus \delta(s_k))) = 1\right],$$

and the *effect* is $\phi(\tau, k, \delta) = p_\delta(\tau, k) - p_0(\tau, k) \in [-1, 1]$.

- Both rates are estimated by N independent rollouts; p_0 is estimated with the original step *re-executed*, not read from the observed outcome, which makes the effect robust to run-to-run stochasticity.
- A step k is α-*decisive* for a failed τ if $\max_{\delta \in \Delta_{\text{fix}}} \phi(\tau, k, \delta) \geq \alpha$, and α-*fragile* for a successful τ if $\min_{\delta \in \Delta_{\text{inj}}} \phi(\tau, k, \delta) \leq -\alpha$.

Definition 2 (Failure attribution)

For a failed trajectory τ the *repair-effect profile* is

$$e_\tau(k) = \max_{\delta \in \Delta_{\text{fix}}} \phi(\tau, k, \delta), \qquad k = 1, \ldots, T,$$

and *failure attribution* is the estimation of e_τ. For a threshold $\alpha \in (0, 1]$ stated with every result, the labels used in the literature are summary statistics of the profile:

- **error set** $E_\alpha(\tau) = \{k : e_\tau(k) \geq \alpha\}$ — the steps whose repair would have rescued the run;
- **decisive step** $k^\star = \min E_\alpha(\tau)$ — the earliest of them;
- **lock-in** $k_{\text{lock}} = \max E_\alpha(\tau) + 1$ — the first step from which no single-step repair recovers the run;
- **responsible agent** $a^\star = a_{k^\star}$ — the agent that emitted the decisive step; no separate "who" label is defined;
- **fault side** — whether the maximising δ at $k^\star$ acts on the output u_k (model-side) or on the observation o_k (tool- or environment-side).

Correspondence with the surveyed labels.

- $k^\star$ is the target of Who&When, TraceElephant, TrajErrBench, LongRCA, CORRECT-Error and the injected datasets, and the t_{err} of Failure-as-a-Process.
- E_α is what MP-Bench's vote counts, AgentRx's failure events and TRAIL's error spans approximate; MP-Bench's per-step consensus rate is a human estimate of $e_\tau(k)$ itself.
- k_{lock} is AgentRx's "first failure from which the run never recovers" and Failure-as-a-Process's t_{lock}; $k_{\text{lock}} - k^\star$ is that paper's fix window (median 1 step).
- $a^\star$: the sender of the labelled step equals the responsible role in 99.2% of LongRCA and 100% of TracerTraj records, so defining the agent through the step loses nothing and removes the naming problems of Section 7.
- Fault side recovers MegaRCA-Mix's model / harness / environment / grader sides and the environment-caused failures that SearchAuditBench excludes.

What the definition is robust to.

- *Annotator variance.* The target is measured, not adjudicated; a dataset whose ground truth depends on who labelled it cannot be a stable target across datasets.
- *Run-to-run stochasticity.* p_0 is re-estimated by re-execution, so a run that failed by chance has a small effect everywhere instead of a forced label.
- *Several causes.* The error set is graded by effect size; MP-Bench's 16% single-step agreement becomes a property of the profile (several steps of comparable effect) rather than label noise.
- *Recovery.* Decisive step and lock-in come from one profile, so the two conventions in the literature are reconciled instead of chosen between.
- *Observation level.* The definition lives on the system, not on the log: output-only logs, LLM-call traces and span trees are different estimators of the same quantity, not different tasks.

What must be fixed for it to be well posed.

- *The vocabulary* Δ (Assumption 2, Table 2). "Had it been correct" is not learnable until "correct" is an operation; results are reported per intervention. Errors with no single-step intervention—a wrong team composition before the log begins, which is where 20 of 126 Who&When Algorithm-Generated labels sit (step 0)—are declared out of scope, not scored as zero.
- *N and α.* Effects are reported with intervals; $N = 5$–10 rollouts separate an effect of 0.5 from 0 but not 0.1 from 0, so α is not set below the resolution that N affords.

Proxies when replay is impossible.

- Live-web GAIA and AssistantBench runs and output-only logs violate Assumption 1. There the human decisive step is used as a proxy for $k^\star$, scored with exact match *and* a $\pm k$ window, and the agreement figures above are quoted as its ceiling.
- No method is evaluated on proxies alone: the replayable environments of the survey (SWE-bench and SWE-Bench Pro, ALFWorld, WebShop, τ^2-Bench, TravelPlanner, VitaBench, Terminal-Bench)

carry the measured version; the proxies test transfer.

In one sentence: failure attribution is the estimation of a step-level repair-effect profile under a declared intervention set, and the decisive step, the error set, the lock-in point and the responsible agent are summary statistics of that profile.

The shared data engine. Every human-labelled dataset of the survey is a test set, and only four of the 21 ship trajectories with both outcomes. The measured parts of the four proposals are therefore *collected*, not downloaded, on the systems of Table 3:

- rows 1–5 are systems whose agent code and environment are public and replayable (Assumption 1), with a verifier that supplies S for free;
- each row names the released human labels that can score a result *on that system*, so that no new annotation is needed;
- row 6 (live web) supports recorded-prefix replay only and is used for transfer tests;
- the same infrastructure—state restore and an LLM-call cache—serves the rollouts of Proposal 1, the corpora of Proposal 2, the restarts of Proposal 3 and the injections of Proposal 4, which is what makes the four proposals one programme rather than four.

Set	Intervention δ	Definition (changes step k only)
Δ_{fix}	ORACLE-ACTION	replace u_k by the action a human/LLM with access to the ground truth deems correct at k
	HINT	prepend to u_k's context a one-sentence statement of the fact the step got wrong (SymTrace's suspicious-node intervention [39])
	FIX-OBSERVATION	replace a corrupted/empty o_k (404, OCR failure, swallowed output) by the correct observation
	REASSIGN	let a different agent $a' \neq a_k$ produce step k
Δ_{inj}	CORRUPT-OUTPUT	rewrite u_k with a context-aware error of mode m (the Aegis/Who&When Pro injectors)
	CORRUPT-OBSERVATION	perturb o_k (wrong number, truncated page, failing tool)
	DELETE	drop step k and continue
	SWAP-AGENT	route step k to an agent not designed for it

Table 2: A starting intervention vocabulary Δ (Assumption 2). Each entry is a deterministic transformation; what varies across rollouts is only the continuation.

System family (public agent code)	State restore for Roll	Verifier S	Released labels usable as test set	Remark
1 τ^2-Bench airline / retail: tool-calling agent + simulated user (TrajErrBench, AgentRx)	reset of domain DB and user simulator; seeded	DB-state constraints	TrajErrBench 400 (κ 0.91), AgentRx 29	cheapest: 29 messages, 20 k characters per run
2 ALFWorld, WebShop: AgentDebug's modular ReAct agent	deterministic text simulators	task reward	AgentErrorBench 100 + 50	30-step cap; observations repeat the page
3 SWE-bench Verified / Pro, Terminal-Bench 2: SWE-Agent with TraceElephant's tracing middleware, OpenHands, mini-SWE-agent, Terminus 2	container snapshot (Harbor)	hidden tests	TraceElephant 44, TrajErrBench 86, Failure-as-a-Process $t_{\text{err}}/t_{\text{lock}}$ for 1,184 runs once re-joined with the CodeTracer release	long: 120–300 steps, 200 k characters; dominates the budget
4 MASLab systems (LLM Debate, MacNet, AgentVerse, DyLAN, SmolAgents, Magentic-One) on GSM8K / MATH / HumanEval / MMLU-Pro: Aegis wrappers and injection factory; MetaGPT / AFlow runs of TracerTraj	no environment state; LLM-call cache, temperature 0	answer match, unit tests	Aegis 600 (agent + mode, no step), TracerTraj test 790 (synthetic)	injector released; the natural setting for Δ_{inj} with absorbed branches kept
5 WebArena (Verified), AgentBench-OS, BIRD: HORIZON agents; LongRCA's planner–surfer–verifier controller	docker reset per task	environment verifier	HORIZON 294 judge step labels, LongRCA WebArena 177	HORIZON agents public; LongRCA system code not released
6 Live-web GAIA / AssistantBench / BrowseComp: Magentic-One, CaptainAgent, smolagents, deep-search scaffolds	none (recorded-prefix replay only)	answer match	Who&When 184, TraceElephant 176, MP-Bench 289, SymFail 536, TRAIL 117, AgentHallu 36, SearchAuditBench 1,243	Assumption 1 fails: transfer tests only

Table 3: The data engine shared by the four proposals: systems of the survey on which unlabelled trajectories can be collected with replay, the verifier that gives the outcome for free, and the released labels that score the result on them. One thousand runs of one system cost about $1{,}000 \times 30$ steps $\times$ 5 k tokens $\approx 1.5 \times 10^8$ tokens with an open model.

3 Proposal 1: Counterfactual perturbation screens on agent trajectories

Motivation. Every dataset in this survey defines the decisive step counterfactually, but only TraceElephant verifies the counterfactual by re-execution, and the injected datasets (Who&When Pro, Aegis, TracerTraj, CORRECT-Error) keep only the injections that produced a failure. Five observations from the raw data make the gap concrete:

- *The root cause is run-specific.* Same task, same system family, re-run: the blamed agent agrees between Who&When and TraceElephant in only 18 of 86 paired runs.
- *Labels are never tested.* In LongRCA, 364 of 1,140 reasons state no causal link to the outcome, and roughly one in five blames "redundant/duplicate" work in runs that still produced a final plan.
- *Absorption is the common case.* Aegis reports that injections flip a successful run into a failure between 34% (DyLAN) and 77% (LLM Debate) of the time; no dataset records the absorbed branch.
- *Re-runs drift.* SymTrace [39] re-ran 54 successful MAS executions three times each and saw 39 of them fail at least once, while unguided reruns of failed executions reproduced the failure in only 67.97% of attempts.
- *Recovery is large.* In the faulty-agent injections of Huang et al. [22] a reviewing Inspector agent recovered up to 96.4% of the injected errors.

Absorption and run-to-run drift are large effects, not noise. The proposal turns attribution from a human-labelled classification target into the *interventional quantity* of Definition 1, measured by rollouts and then predicted.

Closest prior work. Five strands contain parts of the programme; none contains the quantity.

- *Replay-based repair.* SymTrace [39] (§3.7) records a MAS execution, picks an *anchor* node, serves every LLM and tool call before the anchor from the log and lets everything after it run live—the operational form of $\text{Roll}(\tau_{<k} \oplus s'_k)$. Its intervention augments the anchor's request with a category-specific repair instruction (the HINT of Table 2). On SymFail (536 evaluator-confirmed failures of AG2, CrewAI and Magentic-One) this repairs 20.15% of failures in one attempt, against 6.90% for three unguided reruns and 3.7–4.3% for self-reflection and critic-agent baselines.
- *What replay-based repair lacks.* SymTrace runs one temperature-0 continuation per intervention, so it observes a single draw from p_δ; it takes the anchor from a symptom detector rather than estimating which step is worth intervening on; and it has no injection side.
- *Counterfactual attribution.* Causal Agent Replay [45] states the same do-intervention-then-re-execute estimator with confidence intervals and a Monte-Carlo Shapley extension for interacting steps, but validates it only on synthetic structural causal models. Minimal repair families [28] replay *sets* of events and show that single-event replay misses repairs that are only jointly sufficient. EMFA [34] verifies the

candidates of an error-propagation model with counterfactual checks and is the current best step-level attributor on Who&When (+3.45 / +4.40 pp on the two subsets); TraceElephant's dynamic setting (§3.2) is the same idea with a replayable environment.

- *Recovery as a measured property.* Huang et al. [22] inject faulty agents into six MAS and find that a hierarchical topology loses 5.5% of performance versus 10.5–23.7% for flat ones; the hallucination-cascade study of Jamshidi et al. [24] finds net attenuation along three-agent chains (0.422 → 0.272); ToolMaze [63] scores a perturbation-recovery rate separately from task success. None ties recovery to a labelled step of a natural failure.
- *Infrastructure and training.* AgentRewind [67] checkpoints agent context and environment state jointly—an implementation of Assumption 1; Agent-R [58] trains agents on trajectories spliced at the first erroneous step with a correct branch, i.e. it consumes exactly the (prefix, repaired step) pairs that the surrogate of **E 1.2** would predict.
- *Perturbation biology.* Perturb-seq [13] applies a library of CRISPR perturbations to a population of cells and reads out the phenotype of each cell; GEARS [44] and scGen [38] then predict the phenotype of *unseen* perturbations, so that most of the screen runs in silico. Table 4 maps the objects; the correspondence supplies replicate-based effect estimation, graded essentiality and buffering, and interaction effects between two perturbations.

What is missing from all of them is an effect size per (trajectory, step, intervention), estimated from repeated rollouts, on failed *and* successful natural runs.

Perturbation biology	Agent trajectories	Remark
cell with a genetic background	trajectory prefix $\tau_{<k}$ with restored state	the "background" includes the task, the system and the prior errors
CRISPR knockout / activation	DELETE, CORRUPT-$*$ / HINT, ORACLE-ACTION	loss- and gain-of-function interventions on one step
phenotype readout (expression, viability)	success predicate S, optionally the failure mode of the continuation	binary readout today; a richer readout is the new trajectory itself
replicate cells	N independent rollouts	estimates p_δ; biology has thousands of replicates, we have $N \approx 5$–10
essential gene	α-decisive step	graded by effect size, not by a single annotator
genetic buffering / robustness	absorbed injection ($\phi \approx 0$ on a successful run)	the data Aegis and Who&When Pro discard
epistasis (double perturbations)	two-step interventions $(k_1, \delta_1), (k_2, \delta_2)$	tests MP-Bench's claim of several independent causes; cf. minimal repair families [28] and **RQ 4.7**
in-silico perturbation predictor (GEARS, scGen)	surrogate f (Definition 3, **E 1.2**)	replaces most rollouts once calibrated

Table 4: Correspondence between pooled perturbation screens and interventional failure attribution. Where it breaks: a gene library is finite and fixed, whereas edits to a step are open-ended (hence Assumption 2); cells are independent replicates, whereas rollouts share a prefix and a policy, so N is chosen per prefix; a rollout costs tokens rather than reagents.

Setting. The objects are those of Section 2: effect ϕ, profile e_τ, decisive step $k^\star$ and lock-in k_{lock} (Definitions 1 and 2) under Assumptions 1 and 2. Fig. 1 shows the measurement. Two further objects are needed: a model that replaces the rollouts, and a curve that summarises the injection side.

Definition 3 (Effect surrogate)

An *effect surrogate* is a model $f(\tau_{<k}, s_k, \delta) \approx p_\delta(\tau, k)$ that predicts the intervened success rate from the prefix, the step and the intervention *without executing* a rollout. It is trained on Monte-Carlo estimates $\hat{p}_\delta$ and scored as a probabilistic predictor (Brier score, calibration, AUROC of $\{\phi \geq \alpha\}$), separately for $\delta \in \Delta_{\text{fix}}$ and $\delta \in \Delta_{\text{inj}}$.

Definition 4 (Fragility curve and point of no return)

For a successful trajectory τ, the *fragility curve*

$$r(k/T) = \mathbb{E}_{\delta \in \Delta_{\text{inj}}}\, p_\delta(\tau, k)$$

is the probability that an error injected at relative position k/T is absorbed. The *point of no return* is the first k at which r drops below a stated threshold; it is the interventional counterpart of Failure-as-a-Process's t_{lock} (§6.3). Per-agent and per-mode versions of r condition on a_k and on the injected mode m.

Neither object asks for a binary answer: a single re-run is one Bernoulli draw from p_δ, and Section 3.2 shows that draws from the same prefix land on different failures. The topology comparison of Huang et al. [22] and SymTrace's rerun regressions [39] are single points on fragility curves.

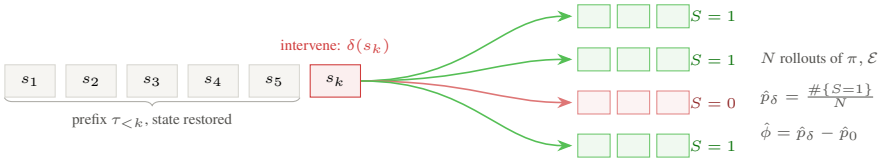


Figure 1: The measured quantity (Definition 1). The state after the prefix is restored, step k is replaced by $\delta(s_k)$, and N independent continuations give a success rate; the same with the original s_k gives the baseline. The effect is the difference.

Research questions of Proposal 1

RQ 1.1 Given a failed trajectory, which steps and which repair interventions raise the success probability of the continuation, and by how much? That is, what is the profile e_τ, and which (k, δ) maximise ϕ?

RQ 1.2 Given a successful trajectory, at which steps does an injected error cascade into failure rather than being absorbed, i.e. which steps are load-bearing?

RQ 1.3 Can the intervention effect be predicted from the prefix, the step and the intervention without executing a rollout, and is the prediction calibrated enough to replace most of the screen?

RQ 1.4 Do effect-based decisive steps agree with the human labels of the surveyed benchmarks, and where they disagree, which side do the rollouts support?

RQ 1.5 How far into a run can an agent system still recover from an injected error, how does that depend on the agent and the error mode, and do verifier agents change it?

Experiments.

E 1.1 — The screen (answers RQ 1.1, RQ 1.2).
Collect natural failed and successful trajectories on the replayable systems of Table 3; for each, intervene at the human-labelled step (failures) or at stratified positions (successes) plus four control positions, with two interventions from each of Δ_{fix} and Δ_{inj} and $N = 8$ rollouts per cell; estimate $\hat{p}_0$, $\hat{p}_\delta$ and $\hat{\phi}$ with intervals. Every continuation is stored with its outcome, which is the raw material of Proposals 2 and 4.

E 1.2 — The surrogate (answers RQ 1.3).
Fit the effect surrogate f of Definition 3 on the output of **E 1.1** and test it on held-out prefixes, held-out tasks and held-out systems. Baselines: process-reward-model value estimates [53], the system's own self-monitoring signals (orchestrator ledger flags precede the labelled step in 24 of 90 Magentic-One traces, §3.2) and per-source position priors.

E 1.3 — Attribution by effect (answers RQ 1.4).
Predict $k^\star = \arg\max_k e_\tau(k)$ from the screen and compare with the human labels of Who&When, TraceElephant, MP-Bench and LongRCA and with the single-rollout counterfactual verifiers of EMFA and Causal Agent Replay [34, 45]. Disagreements between effect-based and human labels, adjudicated by further rollouts, are the interesting output; MP-Bench's three annotators make it possible to ask whether effect size tracks annotator consensus.

E 1.4 — Fragility and lock-in (answers RQ 1.5).
From the injection half of **E 1.1**, estimate the fragility curve $r(k/T)$ of Definition 4 per system, per agent and per injected mode, and its point of no return; compare systems with and without a verifier agent and compare topologies as in Huang et al. [22]. Proposal 4 refines this experiment by keeping the absorbed branches and labelling what happened to the error.

Exp.	Data	Metrics	Baselines / reference values
E 1.1	*Collected*: 200 failed + 200 successful runs on rows 1–5 of Table 3, prefixes restored by snapshot; interventions from Table 2, $N = 8$. Failed runs taken from the released labelled sets on the same systems (TraceElephant SWE-Agent 44, TrajErrBench 486, AgentErrorBench 200) so that a human label exists per run.	effect $\hat{\phi}$ with Wilson intervals; fraction of labelled steps that are α-decisive at $\alpha \in \{0.25, 0.5\}$; fraction of injections absorbed; share of labels outside the vocabulary (declared out of scope)	no baseline (measurement); SymTrace single-rerun repair 20.15% and rerun reproduction 67.97% [39]; Aegis absorption 23–66%
E 1.2	*Train*: $(\tau_{<k}, s_k, \delta, \hat{p}_\delta)$ tuples of **E 1.1**; *test*: held-out prefixes, tasks and systems; transfer to live-web runs (row 6) scored against human labels only	Brier score, expected calibration error, AUROC of $\{\phi \geq \alpha\}$; reported separately for Δ_{fix} and Δ_{inj}; rollouts saved at fixed accuracy	PRM value estimates [53]; ledger flags; position priors (first step, last decision step, per-source histogram)
E 1.3	*Test*: human labels of Who&When, TraceElephant, MP-Bench, LongRCA on the replayable subset; MP-Bench per-step votes	exact and $\pm k$ agreement between $k^\star$ and the human step; nDCG@5 of e_τ against MP-Bench consensus; rank correlation of effect size with vote count; share of disagreements resolved by rollouts in favour of each side	EMFA and Causal Agent Replay single-rollout verifiers [34, 45]; human–human agreement (40.3% LongRCA, α 0.64 TraceElephant) as the ceiling
E 1.4	*Collected*: successful runs of **E 1.1** (TraceElephant withholds its 160 successes; Failure-as-a-Process PASS runs once re-joined)	$r(k/T)$ with intervals per system, agent and mode; point of no return at threshold 0.5; area under r as a scalar robustness score	natural t_{lock} distribution of Failure-as-a-Process; hierarchical-vs-flat loss of Huang et al. [22]; SymTrace rerun regressions

Table 5: Evaluation of Proposal 1: data, metrics and baselines per experiment.

Evaluation: data and metrics. Table 5 lists them per experiment; three conventions apply throughout.

- *Everything is probabilistic.* p_0 for a failed run is not zero (6.90% of failed runs pass within three unguided reruns [39]), so effects carry intervals and the surrogate is scored as a probability, not a classification.
- *Human labels are references, not ground truth.* **E 1.3** reports agreement *and* the adjudicated disagreements; the human–human agreement figures of Section 2 are quoted as the ceiling.
- *Stratify by position and length.* Every curve is reported against k/T, and the rollout budget is stratified by trajectory length so that LongRCA-scale runs do not dominate.

Feasibility with the surveyed data.

- *Environments.* Assumption 1 holds for rows 1–5 of Table 3: SWE-bench / SWE-Bench Pro (TraceElephant's SWE-Agent runs, TrajErrBench), ALFWorld and WebShop (AgentErrorBench), the TravelPlanner and VitaBench sandboxes (LongRCA) and τ^2-Bench with its simulated user (TrajErrBench, AgentRx). Live-web GAIA tests only the surrogate's transfer. SymTrace's recorded-prefix replay admits WebArena-Verified and AssistantBench runs at the price of an unreproducible continuation; AgentRewind's joint checkpoints [67] restore the state after $\tau_{<k}$ cheaply.
- *Injectors.* Injectors for Δ_{inj} exist (Aegis, Who&When Pro, TracerTraj) and only need to be run with the negatives retained.
- *Budget.* 400 trajectories, five positions, two interventions each and $N = 8$ rollouts is ≈25 k continuations; at the tail lengths observed here (medians of 10–145 steps, 30 k–260 k tokens per full trace) this is of the order of 10^9 tokens, affordable with small open models and well below the cost of the human annotation it replaces.

Pitfalls to design for.

- *Stochasticity.* A single temperature-0 replay, as in SymTrace, EMFA and the minimal-repair-family search, is one Bernoulli draw; 39 of 54 successful runs fail at least once in three reruns [39].
- *Exposed downstream errors.* A repair at k can succeed up to step $k + m$ and then fail for a new reason; each rollout is re-attributed (cheaply, by the surrogate) so that "no flip" separates into "wrong step" and "several independent errors".
- *Oracle fixes leak the answer.* ORACLE-ACTION is constructed from the ground truth but must not inject it verbatim; HINT is the weaker, safer variant and both are reported.
- *Vocabulary coverage.* Labelled errors with no single-step intervention (20 of 126 Who&When Algorithm-Generated labels sit on step 0) are declared out of scope rather than scored as zero effect.
- *Cost concentration.* Long-horizon sources dominate the token budget; stratify the rollout budget by trajectory length rather than by trajectory.

What would count as a result.

- A calibrated surrogate (**E 1.2**) that predicts, from the prefix alone, which steps of a failed run are worth repairing and which steps of a successful run are load-bearing.
- Effect-based labels (**E 1.3**) that agree with human consensus where humans agree and resolve the cases where they do not.
- Fragility curves (**E 1.4**) that characterise, for the first time with an interventional definition, how far into a run a multi-agent system can still recover from an error.

The first two reuse the benchmarks surveyed here as test sets; the third is new measurement that none of them currently supports.

4 Proposal 2: Label-free attribution from the flow of trajectories

Motivation. Every human-labelled dataset in this survey is a *test* set; the only training-scale resources are injected (Section 4) and carry the positional skew of Fig. 4. Yeh et al. [55] show that step labels can be dispensed with altogether.

- OAT fits a one-class model of *successful* trajectories—about 100 of them, embedded step by step with an LLM and modelled by a neural controlled differential equation [26]—and flags the steps of a failed run that deviate from the learned dynamics.
- On the authors' own tool-use set it reaches F1 0.44 against 0.21 for

GPT-4o prompting, on Who&When 0.23 against 0.15 for GPT-5, and it runs 200–5 000× faster.
- The signal comes from *selection*, not from the dynamics. The paper's own ablations put an RNN next-step predictor close behind the CDE, and the whole out-of-distribution gain (+0.172 AUROC, at −0.028 in-domain) comes from the gated control path that suppresses atypical inputs. The agent's mistakes are samples from its own policy, so a density model of *all* its trajectories would rate them as typical actions; only the contrast with the success-conditioned flow makes them visible.
- Selecting successes already requires an outcome label per trajectory, after which OAT discards the failed half of the data.
- Successful runs are not clean either: Failure-as-a-Process PASS runs contain 2.3 error events per 22.8 steps, all recovered (Table 6), so "success" is a property of the trajectory, not of its steps.

The proposal asks what the principled estimator is at each level of supervision, down to none, and which errors remain identifiable there.

Closest prior work.

- *One-class trajectory models.* OAT [55] as above; Isolation Forest and one-class SVM on step embeddings are the classical versions and serve as baselines.
- *Monte-Carlo value estimation.* Process-reward models estimate the value of a prefix from the outcomes of sampled continuations [53]; applied to agent runs, this is the rollout-supervised level below.
- *Weak supervision.* Multiple-instance learning [12] with a bag label per trajectory, and positive–unlabelled learning [14, 27] with a known class prior, are the two statistical frames for "some step of this failed run was wrong".
- *Change-point detection.* Bayesian online change-point detection [2] infers a switch between regimes without labels; it is the fully unsupervised level below.
- *Discrete-time survival analysis* supplies the likelihood that ties the step-level hazard to the trajectory-level outcome.

None of these has been fitted to agent trajectories with the outcome as the only supervision, and none has been compared with OAT on the same encoder.

Setting. The notation is that of Section 2. Three objects are introduced: the levels of supervision a corpus can carry, two assumptions about where errors sit, and the hazard model that links steps to outcomes.

Definition 5 (Levels of supervision)

A corpus of trajectories carries one of four levels:
- **L2**: the step label $t^\star$ (every dataset of Sections 3–6);
- **L1**$^+$: the outcomes of several *continuations of the same prefix*—rollouts under Assumption 1, or the two branches of a fault injection—without any human label;
- **L1**: the outcome S only, from the benchmark verifier, unit tests or a judge—free wherever the task has a ground truth;
- **L0**: raw logs, neither S nor $t^\star$.

OAT sits between L1 and L0: it uses S to select the training set and then ignores it. The proposal is built at L1 and L1$^+$.

Assumption 3 (Sparse errors)

The fraction ε of erroneous steps is small in every trajectory, failed or not; Table 6 puts it between 1% and 20% on the surveyed data.

Assumption 4 (Absorption)

A trajectory fails if and only if it contains an error from which the run never recovers; the first such step $t^\star$ determines the outcome, and later steps, right or wrong, are not causal.
- In the terms of Definition 2, $t^\star$ is the lock-in point k_{lock} (AgentRx's first unrecovered failure, Failure-as-a-Process's t_{lock}), not the decisive step $k^\star$, which precedes it by the fix window—median 1 step, but more than 10 in 239 of 1,184 runs (§6.3).
- A model built on this assumption is therefore expected to sit late relative to decisive-step labels and is scored against both.
- Proposal 4 relaxes the assumption by adding a recovery transition.

Definition 6 (Step hazard)

Let $h_t = h_\theta(\tau_{<t}, s_t) \in [0,1]$ be the probability that step t is the first unrecovered error given its prefix. Under Assumption 4 the outcome likelihood is the discrete-time survival model

$$P(S{=}1 \mid \tau) = \textstyle\prod_{t=1}^{T}(1-h_t),$$
$$P(S{=}0,\ t^\star{=}t \mid \tau) = h_t \textstyle\prod_{s<t}(1-h_s),$$

and the attribution is the posterior

$$P(t^\star{=}t \mid S{=}0, \tau) = \frac{h_t \prod_{s<t}(1-h_s)}{1-\prod_s(1-h_s)}.$$

The survival function $\bar{S}_t(\tau) = \prod_{s\le t}(1-h_s)$ is the probability that a run which has reached $\tau_{\le t}$ still succeeds.

Link to Proposal 1. $\bar{S}_t$ is a learned surrogate of the baseline success rate $p_0(\tau, t{+}1)$ of Definition 1, and the online lock-in estimate of **E 2.3** is the point of no return of Definition 4. The two proposals share one object: the rollouts of Proposal 1 measure it, the model here predicts it from the log.

How each level fits h_θ (Fig. 2):
- **L2.** The posterior is supervised directly.
- **L1**$^+$**.** A rollout from $\tau_{<k}$ is an ordinary L1 trajectory that shares its first $k-1$ steps with τ and enters the same likelihood. What it adds is *identification*: two continuations of one prefix with different outcomes place the hazard at or after the branch point, and N continuations give a direct target $\hat{p}_0$ for $\bar{S}_{k-1}$—the Monte-Carlo value estimation of process-reward models [53]. It is the only step-level supervision that costs no label, and it is exactly the by-product of **E 1.1**.
- **L1.** $t^\star$ is marginalised out and θ maximises $\sum_\tau \log P(S_\tau \mid \tau)$ over *all* trajectories: a failed run contributes "some step was fatal", a successful run "no step was". This is multiple-instance learning [12] with the survival link in place of the usual noisy-OR, or positive–unlabelled learning at step level [14, 27] with class prior ε.
- **L0.** S is latent as well. The likelihood then needs a model of the steps themselves and becomes a *change-point mixture*: every trajectory follows a shared pre-change dynamics $q_0(s_t \mid \tau_{<t})$—the dominant mode—up to $t^\star \in \{1,\dots,T,\infty\}$, after which a second regime q_1 applies; $t^\star$ and $S = \mathbb{1}[t^\star{=}\infty]$ are inferred jointly by EM or by Bayesian change-point detection [2].

OAT is the special case in which q_0 is fitted on known successes and the posterior is replaced by a threshold on reconstruction error.

What synthetic data adds. The injectors of Table 2 (Δ_{inj}, released with Aegis, Who&When Pro and TracerTraj) produce from one successful run
- a failed continuation with a known perturbed step, and, as often as not (absorption 23–66% in Aegis), a successful continuation that contains a step known to be wrong;
- both enter the L1 likelihood as ordinary trajectories, so the corpus grows without a label;
- the pair is the sharpest test of the thesis: a hazard model must put high posterior on the injected step of the failed branch and low cumulative hazard on the identical step of the absorbed branch, whereas any detector of deviation from the success flow sees the same step twice and must flag both.

Dataset	Error unit	per trajectory	steps per trajectory	ε
LongRCA	root step	1	156	0.6%
Failure as a Process (FAIL)	t_{err} / error events	1 / 2.5	41.7	2.4% / 6.0%
Failure as a Process (PASS)	recovered error events	2.3	22.8	10.1%
TraceElephant	decisive LLM call	1	27.1	3.7%
Who&When	decisive step	1	22.2	4.5%
AgentRx (public subset)	failure events	4.6	44.7	10.3%
TRAIL	spans carrying $\geq$1 error	577 of 4 626 spans	31.3	12.5%
AgentHallu	responsible step	1	7.5	13.3%
MP-Bench ($\geq$2 of 3 votes)	flagged steps	4.2	23.3	18.0%

Table 6: Step-level error rate ε on the surveyed data (means from the statistics tables of Sections 3–6), the basis of Assumption 3. Even under the most generous labelling, more than 80% of the steps of a failed trajectory are correct, and successful trajectories contain recovered errors at a comparable rate.

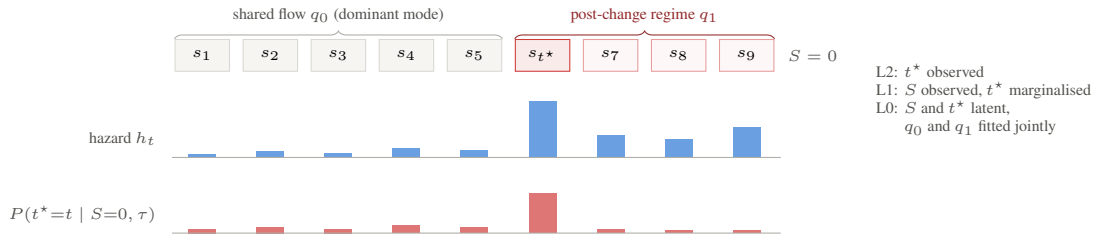


Figure 2: The hazard model (Definition 6). Each step receives a prefix-conditional hazard h_t; the survival product turns the hazards into an outcome probability (what L1 supervises) and, for a failed run, into a posterior over the first fatal step (the attribution). At L0 the same posterior is a change point between a shared dynamics q_0 and a post-error regime q_1.

Research questions of Proposal 2

RQ 2.1 Can a hazard model fitted on all trajectories of a corpus with outcome labels only, and no step label, match or exceed the one-class model OAT and prompted judges on the natural benchmarks when all use the same frozen step encoder?

RQ 2.2 How much does synthetic branching—several continuations of one prefix, or the two branches of an injection—add to step-level identification at zero label cost, and does the hazard model separate the injected step of a failed branch from the identical step of an absorbed branch?

RQ 2.3 Which error classes remain identifiable with no outcome label at all, from the dominant mode of the trajectory flow alone, and which do not?

RQ 2.4 Does the cumulative hazard of a prefix serve as an online monitor with positive lead time to the lock-in point, and how does it compare with the system's own self-monitoring signals?

RQ 2.5 How far does a model built on the absorption assumption sit from decisive-step labels as opposed to lock-in labels, and is the gap the fix window the natural data predict?

Experiments.

E 2.1 — Outcome-supervised hazard at L1 and L1$^+$ (answers RQ 2.1, RQ 2.2, RQ 2.5).
Fit h_θ by maximum likelihood on all trajectories of a corpus with outcomes, using the same frozen step embeddings as OAT so that the comparison isolates the estimator. The estimator is small: a causal GRU or two-layer transformer over the embeddings with a sigmoid hazard head, a sparsity prior on $\sum_t h_t$ (Assumption 3) and the policy decision of Section 2 as the step unit. Steps are discrete events, so no continuous-time machinery is needed; what matters is causality (prefix only) and sparsity. Fit once at L1 and once with the rollout prefixes of L1$^+$ added (the by-product of **E 1.1**), to measure what synthetic branching buys.

E 2.2 — Injected-pair test (answers RQ 2.2).
Run the released injectors on successful runs with the absorbed branches kept, inject uniformly over positions (the released sets do not, Fig. 4) and hold out injected modes. Score the posterior on the injected step of the failed branch against the cumulative hazard on the same step of the absorbed branch; one-class detectors are expected to flag both alike.

E 2.3 — Prefix monitor (answers RQ 2.4).
Because h_t depends on the prefix only, the cumulative hazard $1 - \prod_{s \leq t}(1 - h_s)$ is an online estimate of lock-in. Evaluate its lead time against t_{lock} and against the orchestrator's own ledger flags in Magentic-One traces (§3.2). Proposal 4 (**E 4.7**) adds a delay guarantee to this monitor.

E 2.4 — Fully unsupervised change-point model at L0 (answers RQ 2.3).
Same corpora, outcomes hidden. The question is not whether it beats **E 2.1**—it cannot—but which error classes are identifiable from the dominant mode alone. The generator-likelihood argument predicts that errors visible only in the action stream are not (a hallucinated summary is a typical summary), whereas errors that change the *observation* stream are: tool exceptions, empty pages, repeated queries (MAST FM-1.3), stalled ledgers, premature termination. Fit q_0 on observations and on actions separately.

Exp.	Data	Metrics	Baselines / reference values
E 2.1	*Tier A, measured*: train and test in the same replayable system (rows 1–5 of Table 3); h_θ fitted on self-collected runs, scored on TrajErrBench, AgentErrorBench and the SWE-Agent / SWE-Bench Pro traces; on a subset the $\hat{p}_0$ curves of **E 1.1** are the reference for $\hat{S}_t$. *Tier B, transfer*: frozen model on Who&When (both subsets), MP-Bench, LongRCA, AgentRx and AgentHallu (whose 250 clean runs allow an L1 fit from its own logs).	exact and $\pm k$ step accuracy; distance to t_{err} and to t_{lock} reported separately (**RQ 2.5**); nDCG@5 against MP-Bench consensus; set-F1 over AgentRx events and TRAIL spans; ± 5 window on LongRCA; Brier score and calibration of $\hat{S}_t$ against $\hat{p}_0$ (Tier A only) and of $P(S{=}0 \mid \tau)$	position priors (first step, last decision step, per-source histogram); all-at-once prompting; OAT re-implemented on the same encoder; Isolation Forest and one-class SVM; value regression trained on rollouts only [53] (L1$^+$ without L1)
E 2.2	*Tier C, exact labels*: Who&When Pro text split, CORRECT-Error, TracerTraj test as released; own injections on rows 1–5 with absorbed branches kept, uniform over position, modes held out	posterior mass on the injected step (failed branch) versus cumulative hazard at the same step (absorbed branch); AUROC of fatal-vs-absorbed; accuracy stratified by injected position and by held-out mode	one-class detectors (expected to flag both branches); injector position prior as a control
E 2.3	Failure-as-a-Process FAIL runs (t_{lock}, 1,184) and PASS runs with recovered errors as hard negatives; Magentic-One traces with ledger flags	median lead time to t_{lock}; precision and recall at fixed alarm thresholds; false-alarm rate on PASS runs	Failure-as-a-Process prompted monitor (82% precision, 18% recall, median lead time 0); ledger flags (precede the labelled step in 24 of 90 traces)
E 2.4	same corpora as **E 2.1** with outcomes hidden; taxonomy labels of TrajErrBench modules, Who&When Pro families, TRAIL and AgentRx categories	recall of the labelled step per error category, fitted on observations and on actions separately; the identifiability map is the result	**E 2.1** at L1 as the upper reference; OAT

Table 7: Evaluation of Proposal 2: data, metrics and baselines per experiment. Tiers A–C score the method where its target is measurable, where it must transfer and where labels are exact.

Evaluation: data and metrics. Table 7 lists them per experiment; four conventions apply.

- *Same encoder for every method.* Baselines, OAT and the hazard head share the frozen step embeddings, so differences are differences between estimators.
- *Two references per failed run.* Distance to t_{err} and to t_{lock} are reported separately, because Assumption 4 predicts a late bias against the former.
- *Robustness to outcome noise.* Outcome labels are flipped at rates up to the 12.9% verifier–judge disagreement observed in HORIZON, and judge-only against verifier-only outcomes are compared on the same runs.
- *The injected sets are test sets.* Who&When Pro, CORRECT-Error and TracerTraj are used in Tier C only, never for training, so that the injector's position prior is not learned.

Feasibility with the surveyed data.

- *Released corpora with both outcomes.* AgentHallu (443 hallucinated / 250 clean), HORIZON (2 498 / 1 284 by judge), TRAIL (148 runs not selected by outcome) and Failure-as-a-Process (1 184 / 610, annotations only—the CodeTracer trajectories must be re-joined). The core MAS benchmarks release failures only (TraceElephant withholds its 160 successful runs; Who&When Pro and Aegis do not ship their successful seeds).
- *Collection engine.* Everything else is collected on the systems of Table 3, where the outcome is free; the same state restore and LLM-call cache serve the rollouts of Proposal 1.
- *Evaluation.* The set-labelled datasets for **E 2.1** and **E 2.4** and the three timestamps of Failure-as-a-Process for **E 2.3** exist already; no new annotation is needed.
- *Budget.* One forward pass per step for the embeddings: ≈2 k trajectories × 30 k tokens ≈ 6×10^7 tokens, three to four orders of magnitude below the perturbation screen of Proposal 1; the hazard head trains in minutes.

Pitfalls to design for.

- *The last-step shortcut.* The final answer reveals the outcome, so an L1 model will put hazard on it; the injected datasets show how strong that prior is (median relative position 0.83 in Who&When Pro, 0.95 in CORRECT-Error). Mask the last k steps from the hazard input and report accuracy with the final step excluded.
- *Task difficulty as a confound.* Hard tasks produce both failures and unusual steps, so an outcome model can learn "this task is hard" instead of "this step is wrong". A matched design—the same task run to both outcomes—removes it; the 89 tasks × 21 systems of Section 6.3 are the template.
- *Recovered errors.* The survival likelihood tolerates them (a success only requires that no error was fatal); the plain multiple-instance assumption "success bags are clean" does not. Use the positive–unlabelled formulation with the prior from Table 6 when the any-error target is wanted; Proposal 4 makes recovered errors a target in their own right.
- *Identifiability and model selection at L0.* A change-point mixture can converge to a split that is not error-related (planning phase versus execution phase, say). Use a small L2 set for model selection and calibration only, never for training, and say so.
- *Anomaly is not causality.* Both OAT and **E 2.4** flag deviation; only the counterfactual rollouts of Proposal 1 can confirm that a flagged step is decisive. The two proposals close a loop: the hazard posterior is a cheap proposal distribution for where to intervene, and the rollouts return labels that re-fit h_θ at L2.

What would count as a result.

- An L1 estimator that matches or exceeds OAT and prompting on the natural benchmarks with the same encoder and no step labels, with a calibrated outcome probability as a by-product.
- A measured gain from synthetic branching: the L1$^+$ fit beats the L1 fit on Tier A at equal label cost (zero), and the hazard model separates injected from absorbed branches where one-class detectors cannot.
- An identifiability map for L0: the error categories that can and cannot be recovered from the dominant mode, which tells practitioners what a label-free monitor will and will not catch.
- A prefix monitor with positive median lead time to t_{lock}.

5 Proposal 3: Attribution-guided policy improvement

Motivation. The datasets of this survey are built to score *attributors*. The obvious consumer of a good attributor is the agent that produced the failure: a label $(a^\star, k^\star)$ says which parameters to move and from which state to try again. A literature search on this direction (notes in `notes/attribution_as_training_signal.md`) gives three findings.

- *The mechanism exists, under other names.* The strongest 2026 agent-RL methods locate a "pivotal", "critical" or "irrecoverable" step of a failed rollout and use it to restart rollouts there [17, 32, 36, 47, 49], to reweight the advantage at that step [23, 25, 59, 64, 66] or to build preference pairs there [21, 30, 52, 58]. None cites a failure-attribution benchmark as a source of signal, and none uses an attributor trained on one.
- *The benchmarks never close the loop.* Every downstream experiment inside an attribution paper is inference-time: AgenTracer feeds its tracer's reasoning back as a prompt for three rounds (gains of 4.8–14.2%) [60], AgentDebug re-rolls from the critical step with guidance [65], SearchAuditor resumes at the critical step with a repair directive [33], SymTrace replays one node [39]. No weights change. The only multi-agent blueprint—first-error localisation followed by preference pairs on the blamed agent—is a position paper without experiments [9].
- *Attribution accuracy is never related to training value.* Nobody has asked whether a step metric from Sections 3–6 predicts how much the agent improves when the label is used. The one direct measurement is negative: on 150 SWE-bench Verified runs, LLM-judge "first mistake" labels fall where the success probability does *not* drop in 66% of cases, only 7 of 188 judge answers name the decisive step, and restarting from the judge's step changes expected success by -0.03 to -0.08, whereas restarting from the certified point of no return raises it from 0.44 to 0.53 [46]. SearchAuditor's ablation adds that a generic hint at the predicted step repairs fewer failures (5.6%) than an unguided retry (9.4%), against 17.4% with the auditor's directive [33]: the location is necessary, not sufficient.

The pattern shared by the methods that work is *propose, verify, then credit*: a judge or the policy proposes a step, a few current-policy continuations confirm that the outcome actually turns there, and only then does the step shape credit or data [11, 25, 30, 32]. That verification step is the screen of Proposal 1 with N between 1 and 8.

Closest prior work. Table 8 places the existing methods on the entry points defined below. Four groups:

- *Restart methods* (MiRL, PivoARL, R^{3}L, SPROUT, ELPO) start the next rollouts from a chosen prefix; only ELPO verifies the prefix with rollouts, by binary search for the first irrecoverable step [32].
- *Credit methods* (FAULT, ETRO, ProVer, AgentEvolver, LVRL) reweight or truncate the advantage; ProVer verifies with $N = 8$ continuations but uses positive segments only [25].
- *Data methods* (CSO, Agent-R, CUA-Recovery) build a preference pair or a spliced correction at the step; CSO checks the alternative by policy continuation [30].
- *Multi-agent credit* (MAPPA, C3, AgentGrad) assigns per-agent credit from a coach, from counterfactual continuation of every message, or from sequential intervention, on two- to three-agent mathematics or on prompts only [8, 11, 29]; nothing uses (agent, step) blame on a GAIA-style system.

Setting. The agent is a trainable policy π_θ (single- or multi-agent, each agent $a \in \mathcal{A}$ owning a parameter subset θ_a) run on tasks with a success predicate S; the notation is that of Section 2.

- A group-relative outcome-only update (GRPO and its agentic variants [15]) applies to every step of a failed trajectory the same negative advantage $A_\tau < 0$:

$$g_\tau^{\text{unif}} = A_\tau \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(u_t \mid \tau_{<t}).$$

- Under Assumption 4 only the steps in the error set are causal, so this estimator pushes down $T - |E_\alpha(\tau)|$ correct decisions per failed run; with the error rates of Table 6 that is 80–99% of its negative gradient mass.
- An *attributor* $\mathcal{A}$ maps a trajectory to a non-negative weight vector $\hat{w}_\tau \in \mathbb{R}^T$, an estimate of where the causal mass lies: a point estimate $\hat{w}_\tau = \mathbb{1}[t = \hat{k}]$, a posterior $\hat{w}_\tau(t) = P(t^\star{=}t \mid S{=}0, \tau)$ from the hazard model of Definition 6, or a graded profile $\hat{w}_\tau \propto \hat{e}_\tau$ from the screen of Definition 2.

Definition 7 (Attribution-weighted update)

Given $\hat{w}_\tau$ and a mixing weight $\lambda \in [0, 1]$, the attribution-weighted policy gradient of a failed trajectory is

$$g_\tau^{\hat{w}} = A_\tau \sum_{t=1}^{T} \left[(1-\lambda) + \lambda T \hat{w}_\tau(t)/\|\hat{w}_\tau\|_1\right] \nabla_\theta \log \pi_\theta(u_t \mid \tau_{<t}),$$

so that $\lambda = 0$ recovers g_τ^{unif} and $\lambda = 1$ with the true $w_\tau = \mathbb{1}[t = k^\star]$ concentrates the whole penalty on the decisive step. Successful trajectories are unchanged. The bias of $g^{\hat{w}}$ relative to the ideal g^{w} is governed by the attributor's precision at the chosen tolerance—what the step metrics of this survey measure—and its variance reduction relative to g^{unif} by $1 - \varepsilon$.

Definition 8 (Entry points)

The weight vector can enter training in four places (Fig. 3):

- **R — restart.** Rollouts for the next update start from $\tau_{<k}$ with $k \sim \rho_\tau \propto \hat{w}_\tau$ instead of from the task prompt. This reshapes the state distribution toward decisions that matter and lets a group of continuations share a prefix, so GRPO's within-group comparison becomes a step-level comparison at k. Needs Assumption 1.
- **C — credit.** The update of Definition 7, optionally with *truncation*: no dense supervision after $\hat{k}$, because later steps are conditioned on a state the current policy would not reach [17, 23]. Needs only the log and an attributor.
- **D — data.** A preference pair or a spliced correction at $\hat{k}$: the policy's action against an alternative from which a continuation succeeds [30], or the failed prefix followed by a successful sibling branch [58]. Needs one rollout from $\tau_{<\hat{k}} \oplus \delta(s_{\hat{k}})$ with $\delta \in \Delta_{\text{fix}}$.
- **M — culprit-only update** (multi-agent). $\hat{w}$ is indexed by (agent, step); the gradient is applied to $\theta_{a^\star}$ and the advantages of the other agents are masked, the weight-space analogue of intervention-guided prompt optimisation [11].

Definition 9 (Training value of an attributor)

For a fixed recipe, budget B of environment steps and seeds, let $\pi_B^{\mathcal{A}}$ be the policy after training with weights from $\mathcal{A}$ and π_B^{unif} the outcome-only policy at the same budget. The *training value* is $V_B(\mathcal{A}) = J(\pi_B^{\mathcal{A}}) - J(\pi_B^{\text{unif}})$, with J the held-out success rate; the *accuracy-to-value curve* is V_B plotted against the attributor's step accuracy at the tolerance used by the recipe.

The attributor ladder. The experiments hold the training recipe fixed and vary only $\mathcal{A}$, ordered by cost and by how much of the outcome it has seen:

1. *position priors*: last decision step, per-source histogram (Proposal 2);
2. *outcome variance, no attribution*: start states where the reference policy sometimes fails [56];
3. *the policy's own reflection* [17];
4. *prompted judge*: the all-at-once protocol of Who&When with a frontier model;
5. *trained attributors*: an 8B model trained on TracerTraj, Aegis or Who&When Pro with the AgenTracer recipe [60], and the hazard model of Proposal 2 fitted at L1 on the same system's runs;
6. *rollout-verified*: any of 3–5 followed by the screen of Proposal 1 with $N \leq 8$ at the proposed step and its neighbours [25, 32];
7. *oracle*: the rollout-measured $k^\star$ with large N, affordable only on a test subset.

Each rung is scored twice: by the attribution metrics of this survey on held-out labels of the same system, and by its training value.

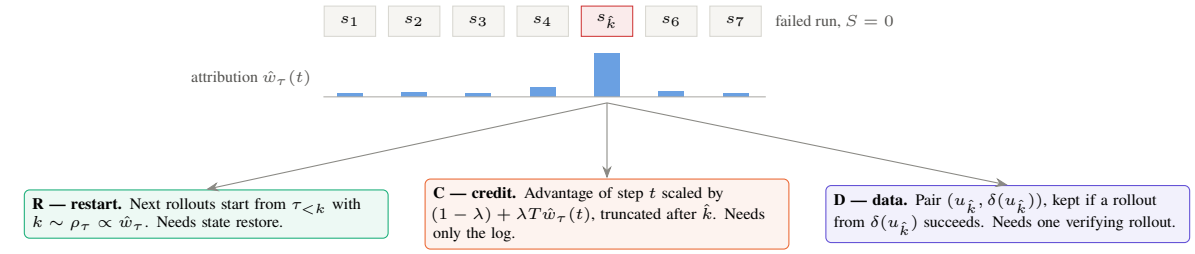


Figure 3: Entry points of an attribution estimate $\hat{w}_\tau$ into agent training (Definition 8). All three consume the same object and differ in what they need from the environment. In a multi-agent system the same vector, indexed by agent and step, also selects whose parameters are updated (entry point M).

Research questions of Proposal 3

RQ 3.1 How does the training value of an attributor depend on its attribution accuracy, and below which precision does attribution-weighted training become worse than outcome-only training at the same budget?

RQ 3.2 Which error of an attributor is the damaging one for training: a label shifted toward the step where the failure becomes visible, or a label that is randomly wrong?

RQ 3.3 Does negative credit concentrated on a rollout-verified decisive step beat unverified judge credit at equal budget, and does it avoid the reward-hacking signature of fixed process rewards?

RQ 3.4 In a multi-agent system, does updating only the blamed agent beat a shared reward, when the blame comes from an attributor trained on the datasets of this survey?

RQ 3.5 When a policy is trained against attributed failures, do the targeted failure modes shrink without the other modes growing?

Method	Attributor (who proposes $\hat{k}$)	Verified?	Entry	Gain vs. outcome-only
MiRL [49]	27B judge, clustered into 8 misbehaviours	replay only	R + judge-gated reward	TB-Lite 51.8 vs 43.4
PivoARL [17]	the policy's own reflection	no	R + C (prefix credit)	83.9 vs 73.9 (GiGPO)
R^{3}L [47]	the policy's own diagnosis	no	R + C (prefix mask)	ALFWorld 0.72 $\rightarrow$ 0.93
SPROUT [36]	policy as reviewer, with test diagnostics	no	R (snapshots)	TB-2.1 64.0 vs 39.3
ELPO [32]	binary-search rollouts: first irrecoverable step	MC	R + C (suffix clip)	60.7 vs 51.2
FAULT [66]	self-diagnoser with step and quote, evidence-checked	quote match	C (priced penalties)	ALFWorld 91.0 vs 49.3
ETRO [23]	27B writer: earliest unrecovered decisive error	no	C (truncated distillation)	WebShop 85.8 vs 73.7
ProVer [25]	agentic judge proposes a segment	MC, $N{=}8$	C (positive only)	$+7$ to $+10\%$ rel.
AgentEvolver [59]	judge labels every step good/bad	no	C ($\alpha{=}0.1$–0.2)	47.6 vs 41.3
LVRL [64]	frozen LLM value function, credit = value drop	no	C	up to $+13.6$ pp
CSO [30]	PRM candidates, expert alternative	MC, policy continuation	D (DPO at $\hat{k}$)	GAIA 49.5 vs 38.9
Agent-R [58]	the policy's own first-error judgement	no	D (splice)	70.7 vs 62.4 (splice at end)
CUA-Recovery [21]	gold per-step lineage, human-checked	yes	D (recovery SFT)	35.2 vs 23.6 (token-matched)
MAPPA [29]	LLM coach scores every (agent, action)	no	M (dense)	no shared-reward ablation
C3 [8]	counterfactual continuation of every message	MC	M (dense)	MATH500 82.8 vs 74.5
AgentGrad [11]	sequential intervention: first agent whose fix flips S	yes	M (prompt only)	$+11.8$ vs $+9.2$ (GEPA)

Table 8: Existing methods placed on the entry points of Definition 8. "Verified" states whether the proposed step is checked against the outcome before it is used. Numbers are the papers' own and are not compute-matched across rows; see `notes/lit_training_signal/` for the per-paper caveats.

Experiments.

E 3.1 — The accuracy-to-value curve (answers RQ 3.1, RQ 3.2). Recipe: entry points R and C (restart from $\hat{k}$, prefix-only credit, truncation after $\hat{k}$) on a GiGPO-style learner [15], 4B policy, matched rollout budget. Every rung of the ladder is trained, plus *synthetically degraded* attributors: the oracle label shifted forward by d steps toward the symptom, and the oracle mixed with random steps at precision p. The output is the curve of Definition 9, its slope, and the precision below which attribution hurts. The evidence of [46] predicts that rung 4 sits below rung 2 on SWE-bench and that the shift d, not the precision p, is the damaging error.

E 3.2 — Verified negative credit (answers RQ 3.3). Entry point C alone, so that nothing is restarted. Compare (i) unverified judge weights [59], (ii) evidence-checked self-blame [66], (iii) the hazard posterior of Proposal 2 and (iv) the rollout-verified profile $\hat{e}_\tau$ of Proposal 1 used as $\hat{w}$, each at fixed λ and at λ annealed toward 0. ProVer uses only positive segments [25]; this experiment is its negative mirror on natural failures.

E 3.3 — Culprit-only updates in a multi-agent system (answers RQ 3.4). Entry point M on the MASLab systems of row 4 of Table 3 (Aegis wrappers, verifiable answers) and, for transfer, Magentic-One on GAIA (row 6, log-only). Blame from the trained attributor (rung 5) and from counterfactual message replay [8]. Conditions: shared reward to all agents, per-agent leave-one-out credit [31], repair-every-agent data [62], culprit-only gradient and culprit-only preference pairs [9].

E 3.4 — Closing the loop (answers RQ 3.5). After each run of **E 3.1**–**E 3.3**, the failures of the trained policy are re-attributed and re-labelled with the failure-mode taxonomies of Section 5 (MAST modes through the Aegis labeller, AgentErrorTaxonomy). Two numbers: how much the targeted modes shrink, which MiRL reports as 43–57% on unseen prefixes [49], and how much the other modes grow—the "whack-a-mole" risk no paper reports.

Exp.	Data	Metrics	Baselines / reference values
E 3.1	*Systems*: rows 1–3 of Table 3 (ALFWorld and WebShop with the AgentDebug agent, τ^2-Bench, SWE-bench with a snapshotting harness); *accuracy reference*: released human step labels on the same systems (AgentErrorBench 200, TrajErrBench 486, TraceElephant 44); three seeds per cell	held-out success rate J; training value V_B (Definition 9) at matched environment steps; the accuracy-to-value curve and its slope; the break-even precision; attribution accuracy of every rung (exact, $\pm k$) on the held-out labels	outcome-only GiGPO at the same budget; variance-selected start states [56]; recovery training forked from the end of failed runs with no localisation [54]; last-node and random-node restart [39]
E 3.2	same systems; natural failures of the current policy; weights from four attributors	J and V_B; the *reward-hacking signature*: the attributor's own score rising while J falls [10, 35, 61]; fraction of negative gradient mass on the decisive step	outcome-only; unverified judge credit [59]; ProVer-style positive-only credit [25]
E 3.3	*Systems*: MASLab systems of row 4 (answer match, unit tests), Magentic-One on GAIA for transfer; *blame reference*: Aegis-Bench (agent, mode) labels and Who&When-style held-out labels on the same system	J; agent-level accuracy of the blame on held-out labels; per-agent change in generation rate; cost in rollouts per update	shared reward; leave-one-out credit [31]; repair-every-agent data [62]; dense per-agent coach credit [29]
E 3.4	failures of every trained policy of **E 3.1**–**E 3.3**, re-labelled with MAST modes (Aegis labeller) and AgentErrorTaxonomy	shrinkage of the targeted modes; growth of the other modes; net change in the mode distribution (total variation)	MiRL's 43–57% reduction on unseen prefixes [49]; the pre-training mode distribution

Table 9: Evaluation of Proposal 3: data, metrics and baselines per experiment. Every comparison is compute-matched on environment steps, not wall-clock, and reports both.

Evaluation: data and metrics. Table 9 lists them per experiment; three conventions apply.

- *Two scores per attributor.* Every rung is scored by the attribution metrics of this survey on held-out labels of the same system *and* by its training value; the relation between the two is the result.
- *Compute matching.* Several headline gains in Table 8 come from baselines trained from the raw backbone or with fewer rollouts; here environment steps are matched and wall-clock is reported alongside.
- *Off-policy fraction.* A corrected or teacher-written step inside a prefix is off-policy for the learner; the fraction of training tokens that are off-policy is reported with every result.

Feasibility with the surveyed data.

- *Environments and labels.* Rows 1–4 of Table 3 supply replayable systems with released step labels on the same system (AgentErrorBench 200, TrajErrBench 486, TraceElephant's 44 SWE-Agent traces) and Aegis-Bench's (agent, mode) labels for the multi-agent case. MiRL releases code, checkpoint and trajectories under Apache 2.0 [49]; PivoARL and C3 release code.
- *Attributors.* AgenTracer-8B's weights are not released (§4.1), so rung 5 is retrained from TracerTraj (3,998), Aegis (9,533) and Who&When Pro text (6,257), all ungated; the hazard model of Proposal 2 is fitted on the runs **E 3.1** collects anyway.
- *Budget.* One RL run at MiRL's scale is 160 updates $\times$ 8 prompts $\times$ 16 rollouts $\approx 2 \times 10^4$ rollouts; **E 3.1** with seven rungs, four degraded variants, three systems and three seeds is $\approx$ 100 runs, 2×10^6 rollouts, between 10^{10} and 10^{11} tokens with a 4B policy depending on the system (MASLab and τ^2 runs are 5–20 k tokens, SWE-bench runs up to 50 k). The verification rollouts of rungs 6–7 add $N \leq 8$ continuations per failed trajectory, a small fraction of that. **E 3.3** on MASLab systems is cheaper (median 10 turns, 5 k tokens).
- *Shared infrastructure.* State restore and the LLM-call cache are the components of Proposal 1's screen; the verified $(\tau, k, \delta, \hat{p}_\delta)$ tuples that **E 3.2** produces are training data for Proposal 2 at level L1$^+$.

Pitfalls to design for.

- *Symptom shift.* Judge labels drift to the step where the failure becomes visible [46]; injected datasets place labels late (Fig. 4); a policy's own reflection is bounded by its own competence. Restart entry points are the most exposed, since a restart after the point of no return explores a dead state. Verify ($N \leq 8$) or anneal λ; never use rung 3 or 4 alone for R.
- *Reward hacking.* Any score the policy can raise without succeeding

will be raised: VPPO's policy split steps to collect prefix reward [35], AgentPRM's process reward rose while success fell from 82% to 70% [10]. Guards used in the literature: conjunction with the outcome ($r = r_o \wedge r_e$ [49]), a small or decaying weight [59, 61], a penalty budget capped below the success–failure gap and no gradient through the diagnosis tokens [66].

- *Baselines that get the gain without attribution.* Variance-selected start states [56], recovery training forked from the *end* of failed runs (Terminal-Bench 34.8 vs 28.0 [54]), implicit step credit from repeated states [15] and SymTrace's last-node and random-node protocol [39]; **E 3.1** includes all of them as rungs or controls.
- *Off-policy prefixes.* RTI gates teacher-written steps by perplexity [7]; R^{3}L strips the guidance before training [47].
- *State restore.* Deterministic replay loses two thirds of terminal prefixes [49]; per-turn snapshots lose none [36]. Row 3 of Table 3 uses snapshots, and live-web rows cannot use entry point R at all.

What would count as a result.

- A measured accuracy-to-value curve, with the attribution precision below which attribution-weighted training is worse than outcome-only training, and the finding of whether symptom shift or random error is the damaging failure of an attributor.
- Verified negative credit at the decisive step that beats unverified judge credit at equal budget and does not exhibit the score-up, success-down signature.
- A culprit-only update that beats the shared reward on a multi-agent system with blame from an attributor trained on the datasets of this survey, which would be the first use of those datasets to improve an agent.
- A closed-loop measurement showing the targeted failure modes shrink without the others growing, which is the practical meaning of "learning from failure attribution".

6 Proposal 4: Error propagation and absorption—the lifecycle of an error

Motivation. Proposals 1–3 treat an error as a point event with a binary consequence: a step is intervened on and the outcome either flips or it does not. The survey data say that what happens *between* the error and the outcome is the common case, not the exception, and that no dataset measures it.

- *Successful runs contain errors.* Failure-as-a-Process PASS runs carry 2.3 error events per 22.8 steps, every one of them recovered, with a detour length and a near-miss flag per run (§6.3); in AgentRx the root cause is not the first annotated failure in 27% of public trajectories and 41% of Magentic-One logs, because the orchestrator recovers from the earlier ones (§3.6).
- *Injected errors are usually absorbed.* Aegis reports that 23–66% of its injections leave the outcome unchanged (§4.2); a reviewing Inspector agent recovers up to 96.4% of injected faulty-agent errors [22]; claim-level hallucinations attenuate rather than amplify along a three-agent chain (0.422 → 0.272) [24]. All four injected datasets of Section 4 discard the absorbed branch.
- *Recovery is fast or not at all.* The fix window $t_{\text{lock}} - t_{\text{err}}$ has median 1 step (239 of 1,184 runs above 10); of the failed runs, 58.7% never notice the error and 34.9% notice it and apply the wrong fix; 28.0% fail silently, with no externally observable signal at all.
- *Recovery is trainable.* Recovery-targeted fine-tuning raises a computer-use agent from 23.6 to 35.2 at matched tokens [21], and a perturbation-recovery rate falls by ≈37% under implicit tool failures while task success moves much less [63].

The hypothesis this proposal makes measurable is that *a robust agent system is one in which an error introduced at step k is detected or corrected by a later step with high probability and short delay, so that the trajectory still succeeds*. Two fields have measured exactly this for fifty years and supply the vocabulary:

- *Dependable computing* [5]: a *fault* (the perturbation) produces an *error* (a wrong internal state) which may or may not become a *failure* (a wrong output); the error can stay dormant, be masked, be detected, be recovered or propagate. Fault injection measures the *coverage* (the probability that an activated fault is handled) and the *error latency* (the delay until it is) [4, 42], and the PIE model factorises failure probability into execution, infection and propagation [51].
- *Ecology and infrastructure engineering*: *resilience* is the capacity to return to the basin of attraction after a perturbation [20], measured by the depth and duration of the performance dip [6].

The programme transfers these quantities to agent trajectories and gives the quantity $P_{\text{detect}}(k{+}i \mid k)$ of the hypothesis a precise home.

Closest prior work. Besides the three recovery studies above [22, 24, 63], five strands contain parts of the programme; Table 10 summarises what each one lends.

- *Error-propagation profiling* of software modules [19] measures, per module, how often a wrong input produces a wrong output (permeability) and how many consumers it reaches (exposure); EMFA [34] applies an error-propagation model to Who&When but fits it to failed runs only.
- *Recoverability in safe RL* [16, 40] defines a state as recoverable if the agent can return to a safe set and learns reversibility from trajectories; the point of no return of Definition 4 is that notion on agent runs.
- *Multi-state event-history analysis* [1, 3, 43] estimates the competing exits of a subject from a state under censoring, which is what an error that is still live when the run ends requires.
- *Branching processes* [18] give the extinction probability of a multi-type population from its mean-offspring matrix, which is the analytic form of "does one wrong message die out".
- *Quickest change detection* [37, 41, 48] gives the detection-delay versus false-alarm trade-off for the online monitor of **E 2.3**.

None of them has been applied to an agent trajectory with a replayable environment, which is what makes every quantity below measurable rather than judged.

Setting. Fix a successful trajectory τ, a step k and an injection $\delta \in \Delta_{\text{inj}}$ (Table 2), and let $\omega \sim \text{Roll}(\tau_{<k} \oplus \delta(s_k))$ be one continuation. Three things can happen to the injected error, at three different lags $i \geq 1$ after k:

- **detection** D: the first lag at which a step of ω *refers* to the inconsistency (an observation contradicts the perturbed content, a verifier flags it, the agent says so)—the only one of the three that is visible in the text, and the quantity $P_{\text{detect}}(k{+}i \mid k) = \Pr[D{=}i]$;
- **recovery** R: the first lag after which the perturbed content no longer governs the run—operationally, deleting $\delta(s_k)$ from the context of every step after $k{+}R$ leaves the continuation distribution unchanged, and the success probability of the prefix has returned to its unperturbed level;
- **lock-in** L: the first lag from which no single-step repair in Δ_{fix} rescues the run—Definition 2 applied to ω.

Three facts about these times shape every definition below:

- R and L compete: at most one of them occurs, and a run can also end with the error live but harmless (success without recovery), which is a *censored* observation of both.
- D is not an exit: a detected error can still lock in (Failure-as-a-Process's "aware, wrong fix", 34.9%), and an error can be recovered without ever being detected (a second retrieval returns the right value and the wrong one is forgotten).
- Failure-as-a-Process's t_{obs}, t_{lock} and the detour length of its PASS runs are the natural-failure versions of D, L and R; here they are measured on injected errors where k is known by construction.

Field	Model	Reading on agent trajectories	What it yields
Dependable computing [4, 5, 51]	fault / error / failure chain; coverage and latency; PIE	the fate taxonomy of Definition 10	robustness decomposed into execution, infection, propagation
Software reliability [19]	error permeability and exposure per module	per-agent (g_a, π_a, c_a, x_a), matrix M_{ab} (Definition 13)	which agent to harden, which edge needs a check
Event-history analysis [1, 3, 43, 50]	competing risks, multi-state and recurrent-event models, frailty	states live / recovered / locked in (Definition 11); several errors per run	F_{rec}, F_{lock}, covariate effects, per-agent random effects, with censoring handled
Branching processes [18]	multi-type Galton–Watson, $\rho(M)$	wrong message as an infection on the agent graph	analytic absorption probability; topology comparison
Ecology and infrastructure [6, 20]	basin of attraction; resilience triangle	V_t as the state, lock-in as leaving the basin (Definition 12)	critical severity $\sigma^\star$; depth and rapidity as separate numbers
Sequential analysis [37, 41, 48]	CUSUM, SPRT, quickest change detection	online monitor on the prefix (**E 2.3**)	detection delay against false-alarm rate, with a lower bound
Time series [2, 57]	hidden semi-Markov and switching models	latent on-track / derailed regime that can switch back	label-free recovery segments; Proposal 2's change point is the no-return case
Safe RL [16, 40]	recoverability and reversibility of states	point of no return as an irreversible state	learned lock-in predictors with a definition

Table 10: Mathematical models that the lifecycle programme borrows, and what each one contributes.

Definition 10 (Error fate)

The *fate* of the injected error in ω is one of:
- *dormant*: no later step consumes $\delta(s_k)$ (deleting it from every later context changes nothing);
- *masked*: consumed, but the next decision is the same as under s_k;
- *corrected*: $D < \infty$ and $R < \infty$;
- *overwritten*: $R < \infty$ and $D = \infty$;
- *tolerated*: consumed, never recovered, $S{=}1$;
- *fatal*: $L < \infty$, $S{=}0$.

The *coverage* of the system at (τ, k, δ) is $C(\tau, k, \delta) = \Pr[\text{fate} \neq \text{fatal}] = p_\delta(\tau, k)$ of Definition 1; the fate distribution is its decomposition. Only *corrected* and *overwritten* are the hypothesised capability; *dormant* and *masked* are task slack.

The fragility curve of Definition 4 reports only C, i.e. the product of the three PIE factors. Two systems with the same $r(k/T)$ can differ entirely in how they get it: one by dormancy (the task can be solved without the corrupted content), the other by correction. The fate distribution separates the two.

Definition 11 (Error lifetime)

Let $Z_i \in \{\text{live}, \text{recovered}, \text{locked in}\}$ be the state of the error at lag i, with $Z_0 = \text{live}$. The *cause-specific hazards*

$$h_{\text{rec}}(i \mid \tau, k, \delta) = \Pr[R{=}i \mid Z_{i-1}{=}\text{live}],$$
$$h_{\text{lock}}(i \mid \tau, k, \delta) = \Pr[L{=}i \mid Z_{i-1}{=}\text{live}]$$

and the survival of the live state $\bar{S}_i^{\text{err}} = \prod_{j \le i}(1 - h_{\text{rec}}(j) - h_{\text{lock}}(j))$ give the *cumulative incidences* $F_{\text{rec}}(i) = \sum_{j \le i} \bar{S}_{j-1}^{\text{err}} h_{\text{rec}}(j)$ and $F_{\text{lock}}(i)$ likewise.
- $F_{\text{rec}}(\infty)$ is the recovery probability and $\mathbb{E}[R \mid R < \infty]$ the mean time to recovery.
- The same construction with D in place of R gives the detection-latency distribution $P_{\text{detect}}(k{+}i \mid k)$.
- The error is the *subject* of the survival analysis, which the step of Definition 6 is not: Proposal 2's model is the special case $h_{\text{rec}} \equiv 0$ (Assumption 4). The multi-state model of Fig. 4 contains both.

Definition 12 (Resilience loss)

Let $V_t^0 = p_0(\tau, t{+}1)$ be the success probability of the unperturbed prefix (the survival function $\bar{S}_t$ of Proposal 2) and V_t^δ the same for ω. The *resilience loss*

$$\mathcal{R}(\tau, k, \delta) = \sum_{i \ge 0} \left[V_{k+i}^0 - V_{k+i}^\delta\right]$$

is the area between the two curves [6]; its *depth* $\max_i(V_{k+i}^0 - V_{k+i}^\delta)$ and its *rapidity* (the first i at which the gap closes to α) are the robustness and the recovery speed of the system, as two numbers rather than one. Recovery shows as a dip and a rise of V^δ; lock-in as a dip without a rise.

Definition 13 (Per-agent propagation profile)

For an agent $a \in \mathcal{A}$ of a multi-agent system, following [19], four rates—*generation*, *permeability*, *containment* and *exposure*:

$$g_a = \Pr[u_t \text{ wrong} \mid a_t{=}a, \text{ clean input}],$$
$$\pi_a = \Pr[u_t \text{ wrong} \mid a_t{=}a, \text{ live error in input}],$$
$$c_a = \Pr[\text{flags or fixes it} \mid a_t{=}a, \text{ live error in input}],$$
$$x_a = \mathbb{E}[\#\text{ later steps consuming } u_t \mid a_t{=}a].$$

- "How prone is each agent to committing *to* errors" is π_a; "how prone to committing errors" is g_a; a robust team has low π and high c on the agents with high x.
- The matrix $M_{ab} = \pi_b \cdot \mathbb{E}[\#\, b\text{-steps consuming one } a\text{-output}]$ is the mean-offspring matrix of a multi-type branching process [18]: $\rho(M) < 1$ means a single wrong message dies out with probability one, and the extinction probability solves the usual fixed-point equation, so absorption can be *predicted* from pairwise measurements and compared with the empirical C. The topology result of Huang et al. [22] (hierarchical 5.5% loss versus 10.5–23.7% for flat) is a statement about $\rho(M)$.

Checkpoint null model. Let $\mathcal{C}(\tau)$ be the steps whose role is verification (verifier-agent turns, test runs, re-retrievals, orchestrator ledger updates), each with coverage c_j. The null model

$$C(\tau, k, \delta) \approx 1 - \textstyle\prod_{j \in \mathcal{C},\, j > k}(1 - c_j)$$

predicts that $r(k/T)$ is a step function aligned with the checkpoints rather than a smooth decay, that removing a verifier moves it by a measurable c_j, and that the late-position skew of the injected datasets (Fig. 4) is the regime with no checkpoint left.

Relation to Proposals 1–3.

- The screen of Proposal 1 produces the rollouts; **E 4.1** adds the consumption test and keeps the branches, so the marginal cost over **E 1.1** is one LLM call per continuation step.
- Proposal 2's hazard model is the $h_{\text{rec}} \equiv 0$ case of Fig. 4; the multi-state version fits the PASS runs that Table 6 shows to be contaminated, and the recovered errors that Proposal 2 lists as a pitfall become a labelled target (**E 4.6**).
- For Proposal 3, the fate labels change the credit: a recovered error should not carry the negative advantage of a fatal one, and the recovering step is a positive example of the capability the hypothesis names, which recovery-targeted training already exploits without localising it [21, 54].

Research questions of Proposal 4

RQ 4.1 What are the cumulative incidences of recovery and lock-in, and the detection-latency distribution, as functions of the lag after an injected error, and how do they depend on position, error mode, fault side and system? Is the one-step median fix window of natural failures also the recovery time of injected errors, or is it a selection effect of failed runs?

RQ 4.2 What fraction of absorption is dormancy, masking, correction and overwriting, given that only the last two are the hypothesised capability and the first two are task slack?

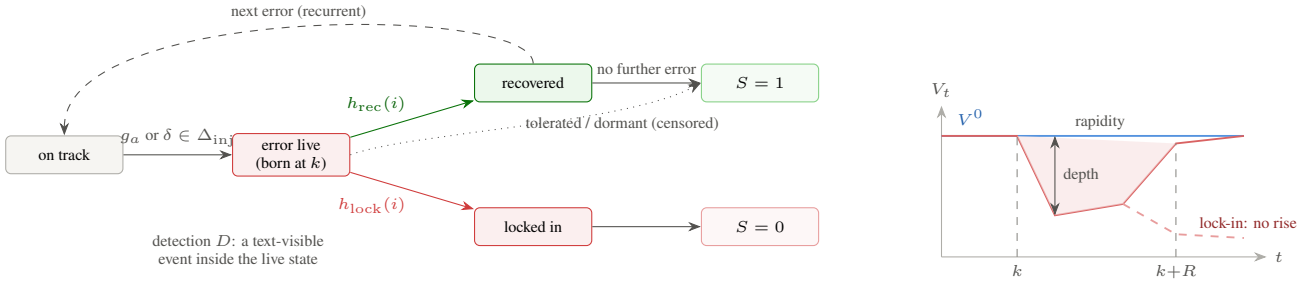


Figure 4: Left: the multi-state model of an error's lifetime (Definition 11). The live state is exited by recovery or lock-in with competing hazards; a run that ends with the error live but harmless is a censored observation. Proposal 2's hazard model is the special case without the recovery transition. Right: the same on the success-probability curve V_t of the prefix (Definition 12): recovery is a dip followed by a rise, lock-in a dip without one, and the shaded area is the resilience loss $\mathcal{R}$ with its depth and rapidity.

RQ 4.3 Is the fragility curve a step function aligned with the system's verification checkpoints, and does removing or adding a verifier move it by the coverage the checkpoint model predicts?

RQ 4.4 How often is an error recovered without ever being detected, and does a detected-but-unfixed error look different in the text from one that was never detected?

RQ 4.5 What are the generation, permeability, containment and exposure rates of the agent roles in the surveyed systems, is the spectral radius of the propagation matrix below one, and does the branching-process absorption probability predict the measured coverage?

RQ 4.6 How does the probability of a fatal outcome grow with the severity of an injected error, and is there a critical severity per position—the basin size of the resilience picture?

RQ 4.7 For two injected errors, is the joint effect independent, compounding, or protective, in the sense that the first error triggers a check that catches the second?

RQ 4.8 What does a recovered error cost in steps and tokens, and is recovering in place cheaper than restarting from the prefix before the error?

RQ 4.9 Can a recovered error be found from the log alone, without rollouts, from the dip-and-rise signature of a learned success-probability curve or from a hidden state that switches back, and with what delay can an online monitor tell a transient dip from a lock-in?

Experiments.

E 4.1 — Fate screen (answers RQ 4.2, RQ 4.4; supplies RQ 4.1, RQ 4.8).
The injection half of **E 1.1** with every continuation *kept* and labelled with Definition 10. The consumption test is a second, cheap counterfactual: for each step $k+i$ of ω, re-issue that single LLM call with $\delta(s_k)$ removed from its context and compare the output (one call per step, not a rollout). Detection D is a judge pass over the text with the injection known; recovery R is the first lag at which the consumption test stays negative for the rest of the run and V^δ has returned to $V^0 - \alpha$; lock-in L is estimated by the surrogate of **E 1.2** and verified by rollouts on a subset. Output: the fate distribution per system, mode, position and fault side, the triples (D, R, L) with censoring indicators, and the resilience loss $\mathcal{R}$ with its depth and rapidity.

E 4.2 — Lifetime curves (answers RQ 4.1, RQ 4.8).
On the output of **E 4.1**, Aalen–Johansen cumulative incidences [1] for recovery and lock-in, then a multi-state Cox model [3, 43] with position, mode, fault side, system and checkpoint distance as covariates, and a shared frailty per agent and per task [50]. Validation on natural labels: $t_{\mathrm{obs}} - t_{\mathrm{err}}$ and $t_{\mathrm{lock}} - t_{\mathrm{err}}$ of the 1,184 Failure-as-a-Process failures, the detour length and recovery count of its 610 passes, and AgentRx's recovered events.

E 4.3 — Checkpoint ablation (answers RQ 4.3).
Align $r(k/T)$ with $\mathcal{C}(\tau)$ in systems with an explicit verifier (Magentic-One's ledger, LongRCA's verification expert, the test runs of SWE agents), then remove, add or delay the verifier and re-run **E 4.1**. The checkpoint model predicts the shift; the alternative—recovery driven by observations rather than by designated checks—predicts none.

E 4.4 — Propagation matrix (answers RQ 4.5).
On the MASLab systems of row 4 of Table 3, inject at the output of agent a and judge the wrongness of each consuming agent's next output against the unperturbed branch; estimate M_{ab}, $\rho(M)$ and the branching-process absorption probability, and compare with the measured C. The same data give (g_a, π_a, c_a, x_a) per role.

E 4.5 — Dose and interaction (answers RQ 4.6, RQ 4.7).
Severity-graded injectors δ_σ (size of a numeric error, fraction of a page truncated, number of corrupted arguments) and paired injections at (k_1, k_2) on the same prefixes as **E 4.1**; report $\sigma^\star(k)$ and the interaction term $\log p_{\delta_1\delta_2} - \log p_{\delta_1} - \log p_{\delta_2} + \log p_0$.

E 4.6 — Recovered-error detection from logs (answers RQ 4.9).
Train on outcome-only data (level L1 of Definition 5) a hidden semi-Markov model [57] with an "error live" state that can switch back, and a value curve V_t with a temporal-difference consistency loss, and score recovery segments against the four released sources of recovered errors: Failure-as-a-Process passes, AgentRx non-root events (§3.6), TRAIL traces that end correctly but carry error spans (§5.2) and MP-Bench steps flagged by one annotator of three (§3.3).

E 4.7 — Online monitor with a delay guarantee (answers RQ 4.9).
A CUSUM [41] on the hazard or value sequence of **E 2.3**, scored by lead time to t_{lock} and by false alarms on passes with recovered errors, the hardest negatives; the Lorden delay–false-alarm bound [37] gives the curve a monitor cannot beat.

Exp.	Data	Metrics	Baselines / reference values
E 4.1	*Collected*: the injected continuations of **E 1.1** on rows 1–5 of Table 3, kept whether or not they fail, injected uniformly over position; one extra LLM call per continuation step for the consumption test; a judge pass for D	fate distribution per system, mode, position and fault side; (D, R, L) with censoring indicators; inter-judge agreement on D (κ); dormant fraction per injector; $\mathcal{R}$, depth and rapidity	Aegis absorption 23–66% as the undecomposed C; Inspector-agent recovery 96.4% [22]; cascade attenuation 0.422 → 0.272 [24]
E 4.2	output of **E 4.1**; *natural validation*: Failure-as-a-Process FAIL ($t_{\mathrm{err}}/t_{\mathrm{lock}}/t_{\mathrm{obs}}$, recovery attempts, 1,184) and PASS (first error step, recoveries, detour length, 610); AgentRx recovered events	$F_{\mathrm{rec}}(i)$, $F_{\mathrm{lock}}(i)$ and P_{detect} with confidence bands; hazard ratios of the Cox covariates; frailty variance per agent and task; agreement in *shape* between injected and natural curves	natural fix-window distribution (median 1, 239 runs > 10); awareness classes (58.7% unaware, 34.9% wrong fix)
E 4.3	systems with an explicit verifier (Magentic-One ledger, LongRCA verification expert, SWE-agent test runs), with the verifier removed, added or delayed	shift of $r(k/T)$ at the checkpoint positions; estimated c_j per checkpoint; goodness of fit of the step-function model against a smooth decay	checkpoint null model versus the observation-driven alternative (no shift)
E 4.4	MASLab systems of row 4 (LLM Debate, MacNet, AgentVerse, DyLAN, Magentic-One); injections at each agent's output; judge of downstream wrongness against ground truth and unperturbed samples	(g_a, π_a, c_a, x_a) per role; M_{ab} and $\rho(M)$; predicted versus measured absorption probability; ranking of agents to harden	hierarchical-vs-flat loss of Huang et al. [22]; Aegis-Bench (agent, mode) labels for g_a
E 4.5	same prefixes as **E 4.1**; severity ladder with 4–5 levels per injector; pairs (k_1, k_2) at fixed distances	$\Pr[\text{fatal} \mid \sigma]$ and $\sigma^\star(k)$ at threshold 0.5; interaction term with its sign and interval	independence (zero interaction); minimal repair families [28] for the repair side
E 4.6	*train*: outcome-only corpora of Proposal 2; *test*: Failure-as-a-Process passes (610), AgentRx non-root events, TRAIL correct-ending traces with error spans, MP-Bench one-of-three steps	recall of recovered errors and of fatal errors reported separately; segment overlap (IoU) with annotated detours; precision on clean passes	OAT and the one-class detectors (expected to flag recovered and fatal errors alike); Proposal 2's hazard model without the return transition
E 4.7	Failure-as-a-Process FAIL (t_{lock}) and PASS runs with recovered errors; Magentic-One ledger traces	detection delay versus false-alarm rate (the full curve); median lead time to t_{lock}; false alarms on recovered-error passes	Lorden bound [37]; Failure-as-a-Process prompted monitor (82% precision, 18% recall, lead time 0); **E 2.3** without CUSUM

Table 11: Evaluation of Proposal 4: data, metrics and baselines per experiment.

Evaluation: data and metrics. Table 11 lists them per experiment; three conventions apply.

- *Survival estimators, not averages.* Every lifetime quantity is estimated with censoring handled (Aalen–Johansen, Cox), because a run that ends with the error live and $S{=}1$ carries no R and no L.
- *Detection is a judgement, recovery is a measurement.* D depends on a judge reading the text; R and L are counterfactual. Inter-judge agreement on D is reported, and D never defines the fate.
- *Natural labels validate shape, not location.* t_{obs} can precede t_{lock} in Failure-as-a-Process and judge labels drift to the symptom [46], so the natural curves are compared with the injected ones in shape.

Feasibility with the surveyed data.

- *Released recovered errors* (for **E 4.2** validation and **E 4.6**): Failure-as-a-Process ships, per PASS run, the first error step, the number of error events and recoveries, the maximal detour length and a near-miss flag, and per FAIL run $t_{\mathrm{err}}/t_{\mathrm{lock}}/t_{\mathrm{obs}}$, the number of recovery attempts, forewarning signals and the awareness class; AgentRx annotates every failure event including the recovered ones (up to 55 per trajectory); TRAIL's 148 runs are not selected by outcome; MP-Bench's three votes per step separate consensus errors from minority ones. No new annotation is needed on the natural side.
- *Injected side.* Aegis reports absorption rates but ships no absorbed branches, and Who&When Pro and TracerTraj discard them likewise, so **E 4.1** is a re-run of the released injectors (Aegis's factory, Who&When Pro's injector) with the negatives kept, on rows 1–5 of Table 3; row 4 (MASLab systems, no environment state) is the cheapest and the natural home of **E 4.4**.
- *Budget.* **E 4.1** reuses the ≈25 k continuations of **E 1.1** and adds one call per continuation step ($\approx 10^6$ calls, below the rollout cost); **E 4.2**, **E 4.6** and **E 4.7** are statistical fits on the result; **E 4.3**–**E 4.5** multiply the injection side by the number of ablations and severities (3–5×) on a subset of prefixes.

Pitfalls to design for.

- *Censoring.* Short runs (ALFWorld's 30-step cap) censor heavily; naive fractions over observed events are biased.
- *Dormancy inflates coverage.* An injection into content the task never needs is absorbed trivially; Definition 10 separates it, and the dormant fraction per injector is itself a result about the injectors of Section 4.
- *Baseline stochasticity.* 39 of 54 successful MAS runs fail at least once in three reruns [39], so V^0 is a curve with intervals, and "fatal" is a drop relative to it, not an absolute failure.
- *Wrongness of downstream outputs* (**E 4.4**) needs a reference: the unperturbed branch is stochastic, so the judge compares against the ground truth or a sample of unperturbed continuations, with the same caveat as D.
- *Injector realism and position prior.* The released injectors place errors late (Fig. 4); inject uniformly over positions and stratify every curve by k/T, or the lifetime curves inherit the prior.

What would count as a result.

- The first measured lifetime curves of errors in agent systems: recovery and lock-in incidences by lag, position, mode and system, with the fraction of absorption that is genuine correction rather than task slack.
- A tested checkpoint model: either $r(k/T)$ is a step function that a verifier ablation shifts by the predicted amount, or recovery is observation-driven and verifier agents are not where robustness comes from.
- Per-agent profiles (g_a, π_a, c_a, x_a) for the roles of the surveyed systems and a propagation matrix whose spectral radius predicts absorption, which would turn "which agent is responsible" into "which agent to harden".
- A label-free detector of recovered errors with a measured delay–false-alarm curve for the online monitor, placed against the Lorden bound.

References

[1] Odd O. Aalen and Søren Johansen. An empirical transition matrix for non-homogeneous Markov chains based on censored observations. *Scandinavian Journal of Statistics*, 5(3):141–150, 1978.

[2] Ryan Prescott Adams and David J. C. MacKay. Bayesian online changepoint detection. *arXiv preprint arXiv:0710.3742*, 2007.

[3] Per Kragh Andersen and Richard D. Gill. Cox's regression model for counting processes: A large sample study. *The Annals of Statistics*, 10(4): 1100–1120, 1982.

[4] Jean Arlat, Martine Aguera, Louis Amat, Yves Crouzet, Jean-Charles Fabre, Jean-Claude Laprie, Eliane Martins, and David Powell. Fault injection for dependability validation: A methodology and some applications. *IEEE Transactions on Software Engineering*, 16(2):166–182, 1990.

[5] Algirdas Avižienis, Jean-Claude Laprie, Brian Randell, and Carl Landwehr. Basic concepts and taxonomy of dependable and secure computing. *IEEE Transactions on Dependable and Secure Computing*, 1(1):11–33, 2004.

[6] Michel Bruneau, Stephanie E. Chang, Ronald T. Eguchi, George C. Lee, Thomas D. O'Rourke, Andrei M. Reinhorn, Masanobu Shinozuka, Kathleen Tierney, William A. Wallace, and Detlof von Winterfeldt. A framework to quantitatively assess and enhance the seismic resilience of communities. *Earthquake Spectra*, 19(4):733–752, 2003.

[7] Xi Chen, Ruixue Ding, Zhigaoyuan Wang, Fanrui Zhang, Qiang Zhang, Boli Chen, Shihang Wang, Tianfu Wang, Ailin Ren, Pengyong Wang, Pengjun Xie, and Hui Xiong. Rollout-time intervention: Unlocking agentic reinforcement learning beyond policy limits. *arXiv preprint arXiv:2610.13438*, 2026.

[8] Yanjun Chen, Yirong Sun, Hanlin Wang, Jinghan Wang, Xinming Zhang, Xiaoyu Shen, Wenjie Li, and Wei Zhang. The trace is the state: Exact credit assignment for LLM agent teams. *arXiv preprint arXiv:2603.06859*, 2026.

[9] Chih-Hsuan, Yang, Tanwi Mallick, Le Chen, Krishnan Raghavan, Amal Gueroudji, Ian T. Foster, and Rajeev Thakur. Who gets the reward & who gets the blame? evaluation-aligned training signals for multi-LLM agents. *arXiv preprint arXiv:2511.10687*, 2025.

[10] Sanjiban Choudhury. Process reward models for LLM agents: Practical framework and directions. *arXiv preprint arXiv:2502.10325*, 2025.

[11] Jaewon Chu, Jinwoo Seo, Jaewon Cho, Jeehye Na, Yunyang Xiong, Youngdae Kim, and Hyunwoo J. Kim. AgentGrad: Intervention-guided prompt optimization for multi agent systems. *arXiv preprint arXiv:2609.08572*, 2026.

[12] Thomas G. Dietterich, Richard H. Lathrop, and Tomás Lozano-Pérez. Solving the multiple instance problem with axis-parallel rectangles. *Artificial Intelligence*, 89(1–2):31–71, 1997.

[13] Atray Dixit, Oren Parnas, Biyu Li, Jenny Chen, Charles P. Fulco, Livnat Jerby-Arnon, Nemanja D. Marjanovic, Danielle Dionne, Tyler Burks, Raktima Raychowdhury, Britt Adamson, Thomas M. Norman, Eric S. Lander, Jonathan S. Weissman, Nir Friedman, and Aviv Regev. Perturb-seq: Dissecting molecular circuits with scalable single-cell RNA profiling of pooled genetic screens. *Cell*, 167(7):1853–1866, 2016.

[14] Charles Elkan and Keith Noto. Learning classifiers from only positive and unlabeled data. In *Proceedings of the 14th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 213–220, 2008.

[15] Lang Feng, Zhenghai Xue, Tingcong Liu, and Bo An. Group-in-group policy optimization for LLM agent training. *arXiv preprint arXiv:2505.10978*, 2025.

[16] Nathan Grinsztajn, Johan Ferret, Olivier Pietquin, Philippe Preux, and Matthieu Geist. There is no turning back: A self-supervised approach for reversibility-aware reinforcement learning. In *NeurIPS*, 2021.

[17] Weiyang Guo, Zesheng Shi, Longhui Zhang, Zeen Zhu, Min Zhang, and Jing Li. Agent reinforcement learning via pivotal-aware self-feedback retry. *arXiv preprint arXiv:2607.03702*, 2026.

[18] Theodore E. Harris. *The Theory of Branching Processes*. Springer, 1963.

[19] Martin Hiller, Arshad Jhumka, and Neeraj Suri. EPIC: Profiling the propagation and effect of data errors in software. *IEEE Transactions on Computers*, 53(5):512–530, 2004.

[20] Crawford S. Holling. Resilience and stability of ecological systems. *Annual Review of Ecology and Systematics*, 4:1–23, 1973.

[21] Xu Hu, Haoming Li, Yuxiang Wang, Jinxin Yang, Kangshuo Li, Licheng Wang, Dingyi Kang, Jiayu Song, Tianhao Wang, Zhanhua Hu, Xu Yang, Qiannan Li, Bingzhe Li, and Feng Chen. CUA-recovery: Benchmarking error recovery in long-horizon computer-use agents. *arXiv preprint arXiv:2610.27645*, 2026.

[22] Jen-tse Huang, Jiaxu Zhou, Tailin Jin, Xuhui Zhou, Zixi Chen, Wenxuan Wang, Youliang Yuan, Michael R. Lyu, and Maarten Sap. On the resilience of LLM-based multi-agent collaboration with faulty agents. *arXiv preprint arXiv:2408.00989*, 2024.

[23] Yibin Huang and Conghui Zhu. Distill until divergence: Error-truncated hindsight distillation for agentic RL. *arXiv preprint arXiv:2610.23048*, 2026.

[24] Saeid Jamshidi, Arghavan Moradi Dakhel, Kawser Wazed Nafi, and Foutse Khomh. Hallucination cascade: Analyzing error propagation in multi-agent LLM systems. *arXiv preprint arXiv:2606.07937*, 2026.

[25] Dongwon Jung, Hemanth Neelgund Ramesh, Yifan Wang, Xiaomin Li, Yuexing Hao, Yu Hu, Muhao Chen, Varun Chandrasekaran, Andrzej Banburski-Fahey, and Jaron Lanier. Targeting pivotal decisions for credit assignment in agentic reinforcement learning. *arXiv preprint arXiv:2609.36178*, 2026.

[26] Patrick Kidger, James Morrill, James Foster, and Terry Lyons. Neural controlled differential equations for irregular time series. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[27] Ryuichi Kiryo, Gang Niu, Marthinus C. du Plessis, and Masashi Sugiyama. Positive-unlabeled learning with non-negative risk estimator. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[28] Bingjie Li, Yumeng Song, Zhongming Yao, and Tianyi Li. Localizing emergent failures in agentic AI: Recovering minimal repair families via counterfactual replay. *arXiv preprint arXiv:2608.29228*, 2026.

[29] Ed Li, Junyu Ren, and Cat Yan. Scaling multiagent systems with process rewards. *arXiv preprint arXiv:2601.23228*, 2026.

[30] Mukai Li, Qingcheng Zeng, Tianqing Fang, Zhenwen Liang, Linfeng Song, Qi Liu, Haitao Mi, and Dong Yu. Verified critical step optimization for LLM agents. *arXiv preprint arXiv:2602.03412*, 2026.

[31] Yanming Li, Xuelin Zhang, WenJie Lu, Ziye Tang, Maodong Wu, Haotian Luo, Tongtong Wu, Zijie Peng, Hongze Mi, Yibo Feng, Naiqiang Tan, Chao Huang, Lian Peng, and Li Shen. Who deserves the reward? SHARP: Shapley credit-based optimization for multi-agent system. *arXiv preprint arXiv:2602.08335*, 2026.

[32] Qiao Liang, Yuke Zhu, Chao Ge, Lei Yang, Ying Shen, Bo Zheng, and Sheng Guo. Learning from the irrecoverable: Error-localized policy optimization for tool-integrated LLM reasoning. *arXiv preprint arXiv:2602.09598*, 2026.

[33] Zhixiang Liang, Yifei Liu, Yidan Huang, Haozhe Zhao, Beichen Huang, Jiaqi Wang, Nan Duan, and Qiong Cao. SearchAuditor: Auditing and attributing failures in long-horizon search agents. *arXiv preprint arXiv:2608.05212*, 2026. Code and data: https://github.com/lzzzx666/SearchAuditor.

[34] Jiaqi Liao, Yuanzhao Zhai, Huanxi Liu, Xu Zhang, Zheming Zhuang, Dawei Feng, Bo Ding, and Huaimin Wang. Error-propagation modeling for failure attribution in LLM-based multi-agent systems. *arXiv preprint arXiv:2610.11600*, 2026.

[35] Haolin Liu, Dian Yu, Sidi Lu, Yujun Zhou, Rui Liu, Zhenwen Liang, Haitao Mi, Chen-Yu Wei, and Dong Yu. Save the good prefix: Precise error penalization via process-supervised RL to enhance LLM reasoning. *arXiv preprint arXiv:2601.18984*, 2026.

[36] Mengyang Liu, Weiming Wang, Haixin Nan, Yu Wu, Peize Li, Shihong Huang, Xuning He, Xue Jiang, and Yihong Dong. SPROUT: Scaling agentic reinforcement learning for long-horizon software engineering. *arXiv preprint arXiv:2610.14405*, 2026.

[37] Gary Lorden. Procedures for reacting to a change in distribution. *The Annals of Mathematical Statistics*, 42(6):1897–1908, 1971.

[38] Mohammad Lotfollahi, F. Alexander Wolf, and Fabian J. Theis. scGen predicts single-cell perturbation responses. *Nature Methods*, 16(8): 715–721, 2019.

[39] Zhongwen Luan, Xiaoyu Zhang, Ming Hu, Yue Yang, Jiongchi Yu, and Xiaohong Chen. Repair or resample? rethinking failure debugging in LLM multi-agent systems. *arXiv preprint arXiv:2608.25920*, 2026. v2, 29 Aug 2026. Introduces the SymTrace replay framework and the SymFail dataset; code and data: https://anonymous.4open.science/r/SymTrace-7234.

[40] Teodor Mihai Moldovan and Pieter Abbeel. Safe exploration in Markov decision processes. In *ICML*, 2012.

[41] Ewan S. Page. Continuous inspection schemes. *Biometrika*, 41(1/2):100–115, 1954.

[42] David Powell, Eliane Martins, Jean Arlat, and Yves Crouzet. Estimators for fault tolerance coverage evaluation. *IEEE Transactions on Computers*, 44(2):261–274, 1995.

[43] Hein Putter, Marta Fiocco, and Ronald B. Geskus. Tutorial in biostatistics: Competing risks and multi-state models. *Statistics in Medicine*, 26 (11):2389–2430, 2007.

[44] Yusuf Roohani, Kexin Huang, and Jure Leskovec. Predicting transcriptional outcomes of novel multigene perturbations with GEARS. *Nature Biotechnology*, 42:927–935, 2024.

[45] Jaineet Shah. Causal agent replay: Counterfactual attribution for LLM-agent failures. *arXiv preprint arXiv:2606.08275*, 2026.

[46] Bingyu Shen, Jiaqi Tang, Lan Wei, Yi Sheng, J. Jenny Li, and Boyang Li. Executable prefix values for locating agent failures. *arXiv preprint arXiv:2610.17633*, 2026.

[47] Weijie Shi, Yanxi Chen, Zexi Li, Xuchen Pan, Yuchang Sun, Jiajie Xu, Xiaofang Zhou, and Yaliang Li. R^3l: Reflect-then-retry reinforcement learning with language-guided exploration, pivotal credit, and positive amplification. *arXiv preprint arXiv:2601.03715*, 2026.

[48] Alexander Tartakovsky, Igor Nikiforov, and Michèle Basseville. *Sequential Analysis: Hypothesis Testing and Changepoint Detection*. CRC Press, 2014.

[49] Runchu Tian, Zaid Khan, Zun Wang, Justin Chih-Yao Chen, and Mohit Bansal. MiRL: Error-targeted reinforcement learning for correcting terminal agent misbehaviors. *arXiv preprint arXiv:2610.35697*, 2026.

[50] James W. Vaupel, Kenneth G. Manton, and Eric Stallard. The impact of heterogeneity in individual frailty on the dynamics of mortality. *Demography*, 16(3):439–454, 1979.

[51] Jeffrey M. Voas. PIE: A dynamic failure-based technique. *IEEE Transactions on Software Engineering*, 18(8):717–727, 1992.

[52] Hanlin Wang, Jian Wang, Chak Tou Leong, and Wenjie Li. STeCa: Step-level trajectory calibration for LLM agent learning. *arXiv preprint arXiv:2502.14276*, 2025.

[53] Peiyi Wang, Lei Li, Zhihong Shao, Runxin Xu, Damai Dai, Yifei Li, Deli Chen, Yu Wu, and Zhifang Sui. Math-Shepherd: Verify and reinforce LLMs step-by-step without human annotations. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[54] Wei Xie, Kaituo Feng, Shiming Fu, Yong Liu, Xiangyu Yue, and Shuangjia Zheng. ReTerm: Recovery training for self-improving terminal agents. *arXiv preprint arXiv:2610.18484*, 2026.

[55] Samuel Yeh, Yiwen Zhu, Shaleen Deep, and Sharon Li. Tracing agentic failure from the flow of success. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2026. arXiv:2607.12747.

[56] Junkeun Yi, Damon Mosk-Aoyama, Baihe Huang, Ritu Gala, Charles Wang, Sugam Dipak Devare, Khushi Bhardwaj, Abhibha Gupta, Oleksii Kuchaiev, Jiantao Jiao, Jian Zhang, and Venkat Srinivasan. PivotRL: High accuracy agentic post-training at low compute cost. *arXiv preprint arXiv:2603.21383*, 2026.

[57] Shun-Zheng Yu. Hidden semi-Markov models. *Artificial Intelligence*, 174(2):215–243, 2010.

[58] Siyu Yuan, Zehui Chen, Zhiheng Xi, Junjie Ye, Zhengyin Du, and Jiecao Chen. Agent-R: Training language model agents to reflect via iterative self-training. *arXiv preprint arXiv:2501.11425*, 2025.

[59] Yunpeng Zhai, Shuchang Tao, Cheng Chen, Anni Zou, Ziqian Chen, Qingxu Fu, Shinji Mai, Li Yu, Jiaji Deng, Zouying Cao, Zhaoyang Liu, Bolin Ding, and Jingren Zhou. AgentEvolver: Towards efficient self-evolving agent system. *arXiv preprint arXiv:2511.10395*, 2025.

[60] Guibin Zhang, Junhao Wang, Junjie Chen, Wangchunshu Zhou, Kun Wang, and Shuicheng Yan. AgenTracer: Who is inducing failure in the LLM agentic systems? In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2509.03312; data release https://github.com/bingreeky/AgenTracer.

[61] Lei Zhao, Zongheng Cao, and Xinyu Hu. When fine-grained credit assignment helps and hurts: Multimodal agentic reinforcement learning through video editing. *arXiv preprint arXiv:2610.35497*, 2026.

[62] Wanjia Zhao, Mert Yuksekgonul, Shirley Wu, and James Zou. SiriuS: Self-improving multi-agent systems via bootstrapped reasoning. *arXiv preprint arXiv:2502.04780*, 2025.

[63] Dongsheng Zhu, Xuchen Ma, Yucheng Shen, Xiang Li, Yukun Zhao, Shuaiqiang Wang, Lingyong Yan, and Dawei Yin. When tools fail: Benchmarking dynamic replanning and anomaly recovery in LLM agents. *arXiv preprint arXiv:2606.05806*, 2026.

[64] Kaijie Zhu, Weixiang Yan, Omar Attia, Abbas Kazerouni, Alexander Toshev, and Andrew Szot. LLM value functions for credit assignment in agentic RL. *arXiv preprint arXiv:2610.15840*, 2026.

[65] Kunlun Zhu, Zijia Liu, Bingxuan Li, Muxin Tian, Yingxuan Yang, Jiaxun Zhang, Pengrui Han, Qipeng Xie, Fuyang Cui, Weijia Zhang, Xiaoteng Ma, Xiaodong Yu, Gowtham Ramesh, Jialian Wu, Zicheng Liu, Pan Lu, James Zou, and Jiaxuan You. Where LLM agents fail and how they can learn from failures. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026. arXiv:2509.25370. Code: https://github.com/ulab-uiuc/AgentDebug.

[66] Yihua Zhu, Qianying Liu, Weixu Qiao, Xuan Ren, Weiwei Xu, Wenbo Li, Wei Wang, Ruijia Chen, Xinmiao Luan, Yin Luo, Hao Huang, Xiang Zheng, and Hidetoshi Shimodaira. My FAULT: Self-diagnosis as credit assignment in self-evolving agentic reinforcement learning. *arXiv preprint arXiv:2610.01161*, 2026.

[67] Yu Zhuang, Kefei Chen, Yitong Duan, Shuxin Zheng, Jian Li, and Xu-Yao Zhang. AgentRewind: Recoverable execution for long-horizon LLM agents. *arXiv preprint arXiv:2608.14380*, 2026.